LAPORAN PRAKTIKUM

POSTTEST 8

ALGORITMA PEMROGRAMAN DASAR



Disusun oleh:

Muhammad Khairul Anaam 2609106026

Kelas A2‘26

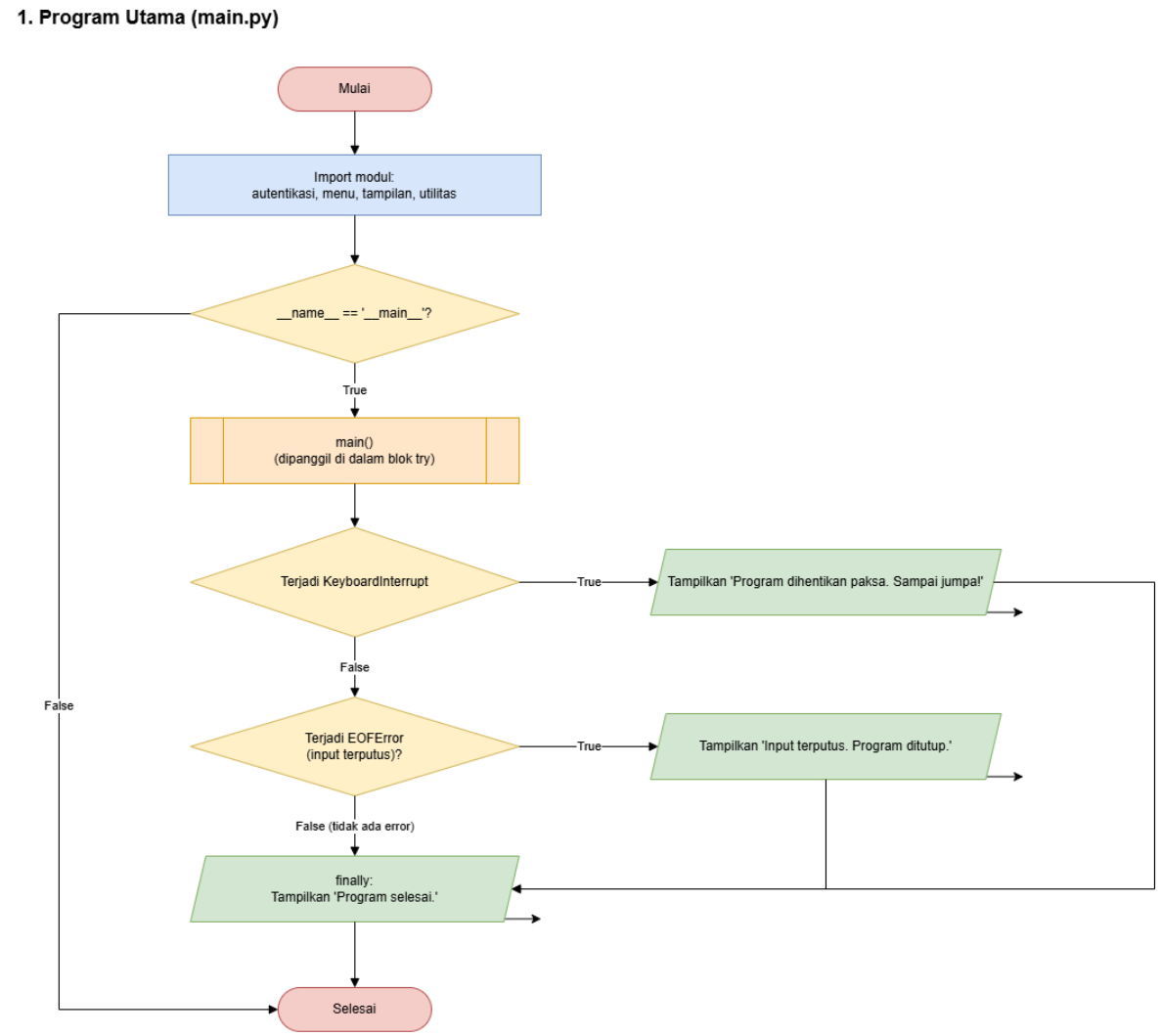
PROGRAM STUDI INFORMATIKA

UNIVERSITAS MULAWARMAN

SAMARINDA

2026

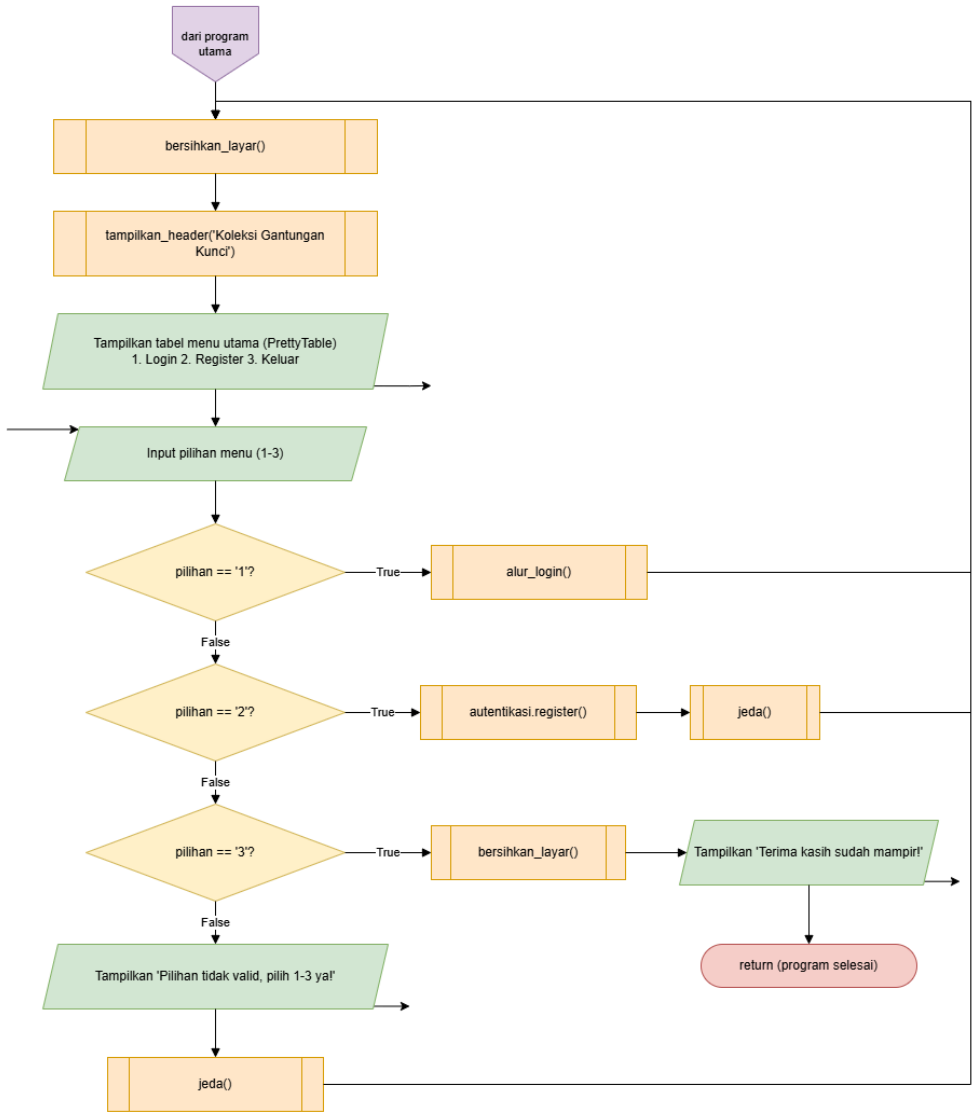
1. Flowchart



Gambar 1.1 Program Utama (main.py)

Program dimulai dengan mengimpor modul, lalu mengecek __name__ == '__main__'. Jika file dijalankan langsung (True), main() dipanggil di dalam blok try. Jika file hanya diimpor (False), program langsung Selesai tanpa menjalankan apa pun. Jika user menekan Ctrl+C atau input terputus (EOFError), pesan yang sesuai ditampilkan. Blok finally selalu mencetak “Program selesai.” apa pun yang terjadi.

2. main() (main.py)

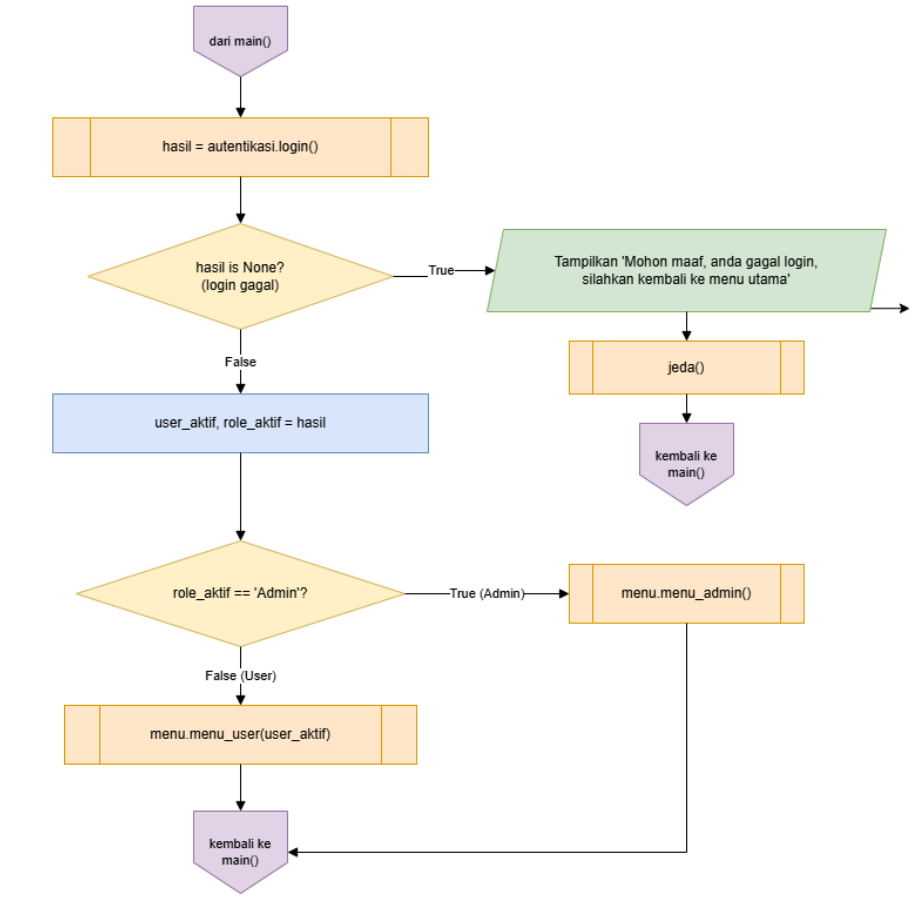


Gambar 1.2 Main()

Ini menu utama yang berulang terus (garis panjang di kanan kembali ke atas). Urutannya: bersihkan layar, tampilkan header, tampilkan tabel menu, lalu input pilihan 1-3.

- Pilihan 1 menjalankan alur_login().
- Pilihan 2 menjalankan register(), lalu jeda().
- Pilihan 3 menampilkan “Terima kasih” lalu keluar dari program.
- Pilihan lain menampilkan “Pilihan tidak valid”, lalu jeda().

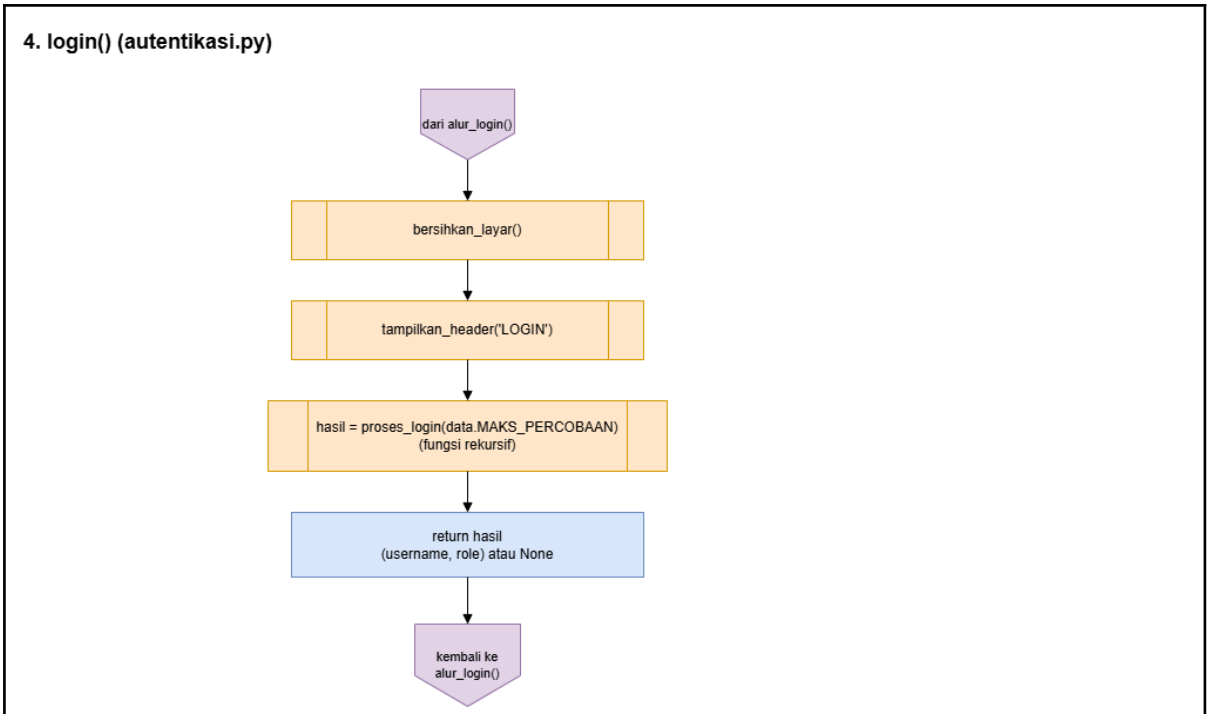
3. alur_login() (main.py)



Gambar 1.3 alur login() (main.py)

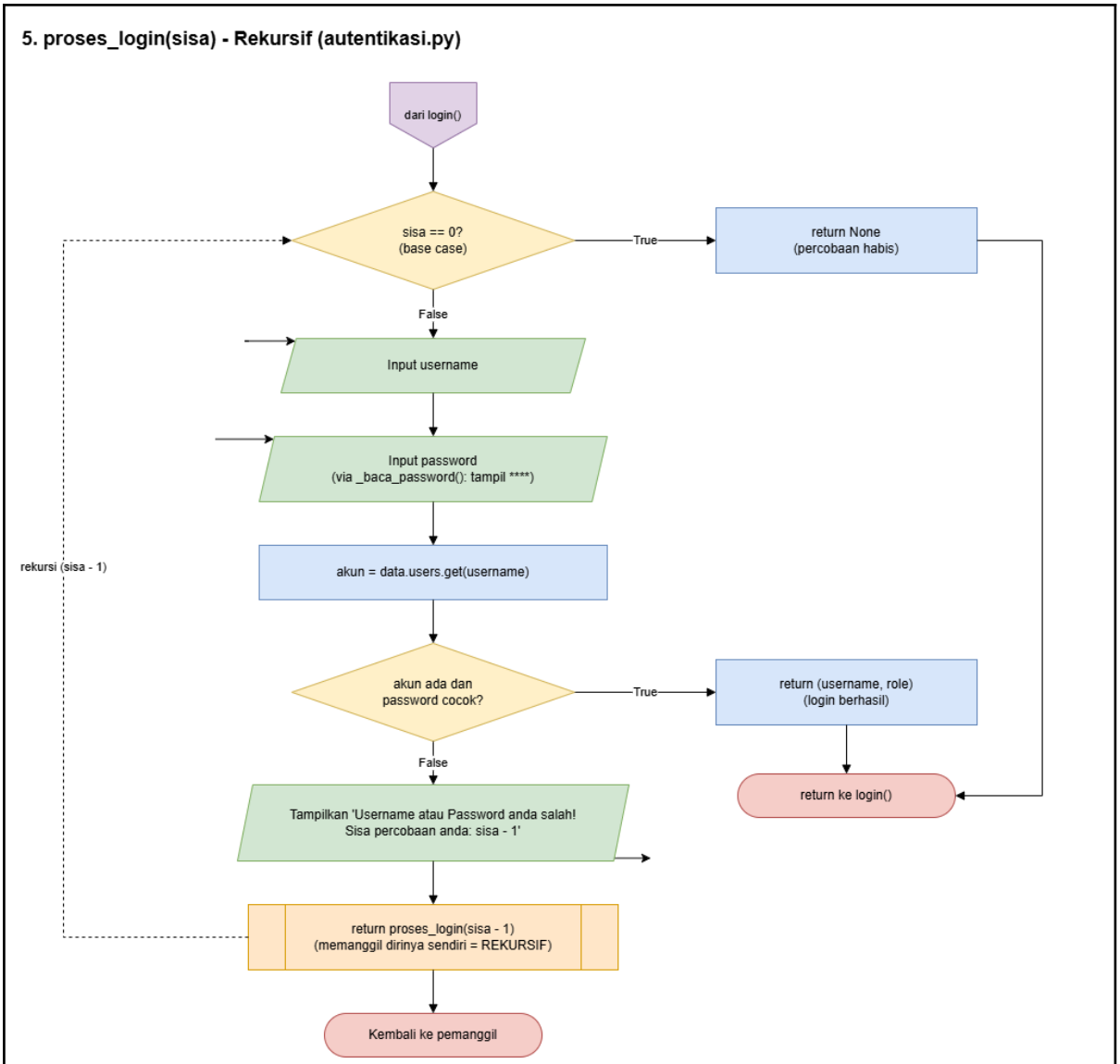
Fungsi ini memanggil autentikasi.login() dan menyimpan hasilnya.

- Jika hasilnya None, login gagal: pesan gagal ditampilkan, lalu kembali ke main().
- Jika berhasil, role dicek. Admin masuk ke menu_admin(), selain itu masuk ke menu_user().
- Setelah logout, alurnya kembali ke main().



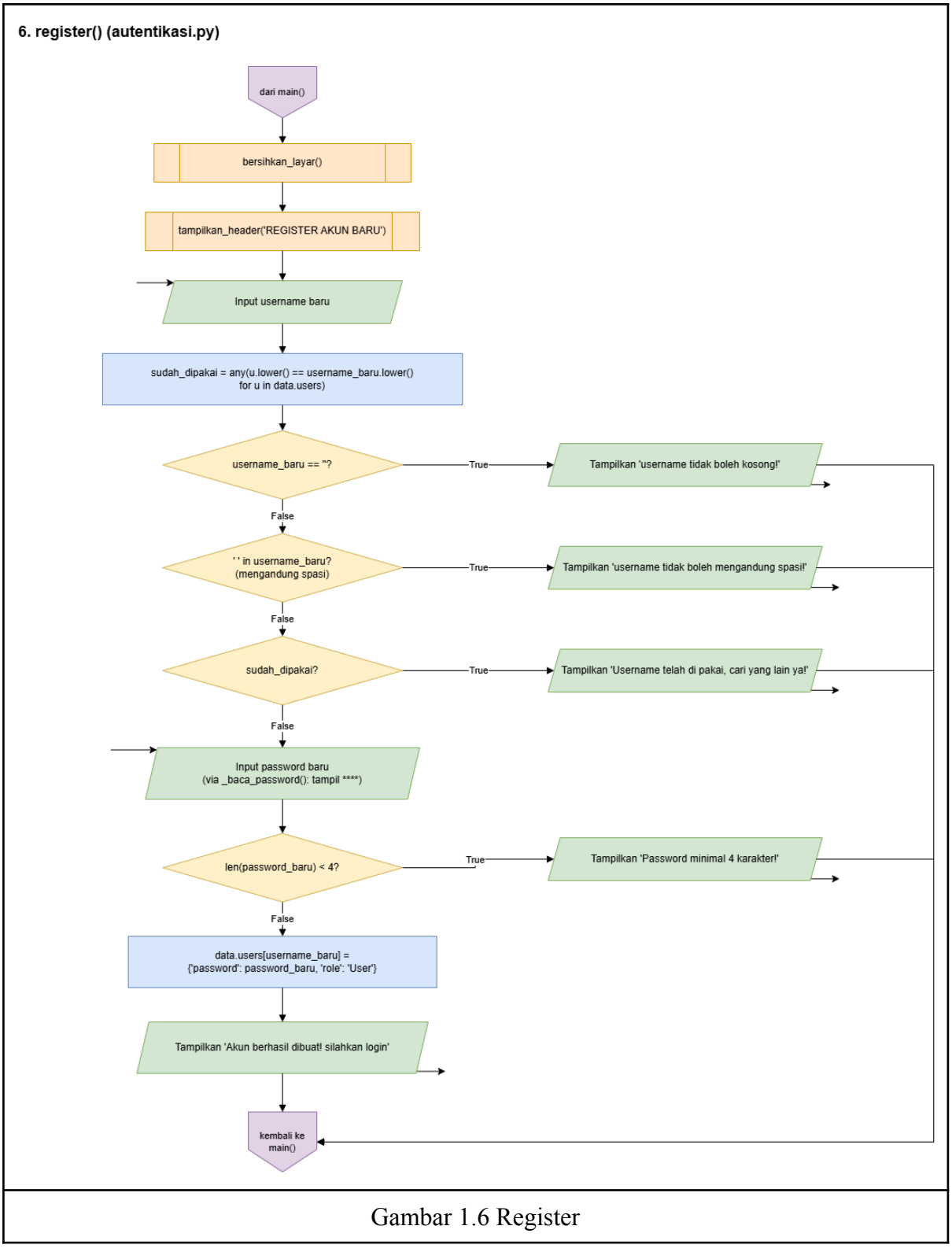
Gambar 1.4 login() (autentikasi.py)

Fungsi ini membersihkan layar dan menampilkan header “LOGIN”. Lalu ia memanggil proses_login(MAKS_PERCOBAAN) dengan batas 3 percobaan, dan mengembalikan hasilnya, yaitu (username, role) atau None.



Gambar 1.5 Proses login

- Base case: jika sisa == 0, fungsi mengembalikan None karena percobaan habis.
- Jika masih ada sisa: user mengisi username dan password (password tampil ****). Akun dicari di data.users.
- Jika cocok: fungsi mengembalikan (username, role).
- Jika salah: pesan “Sisa percobaan anda: sisa - 1” tampil, lalu fungsi memanggil dirinya sendiri dengan sisa - 1. Itulah garis putus-putus yang kembali ke atas. Urutan sisanya 3 → 2 → 1 → 0.



Gambar 1.6 Register

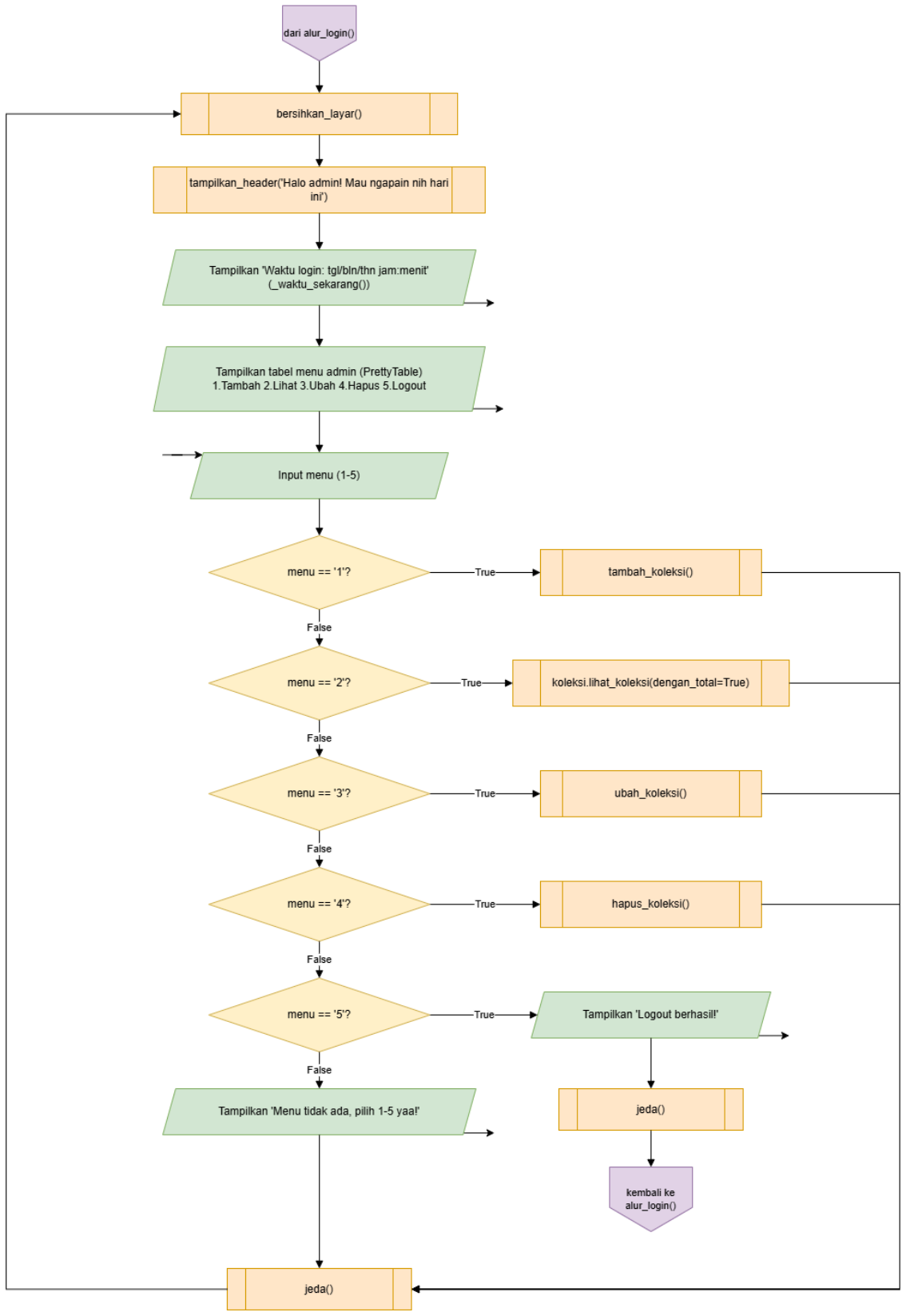
User mengisi username baru. Lalu ada empat pengecekan berurutan:

1. Username kosong.
2. Ada spasi.
3. Sudah dipakai (tidak membedakan huruf besar/kecil).

4. Password kurang dari 4 karakter.

Setiap pengecekan yang gagal menampilkan pesan error, lalu semuanya bertemu di “kembali ke main()” lewat garis kanan. Jika semua lolos, akun disimpan di data.users dengan role “User” dan pesan sukses tampil

7. menu_admin() (menu.py)

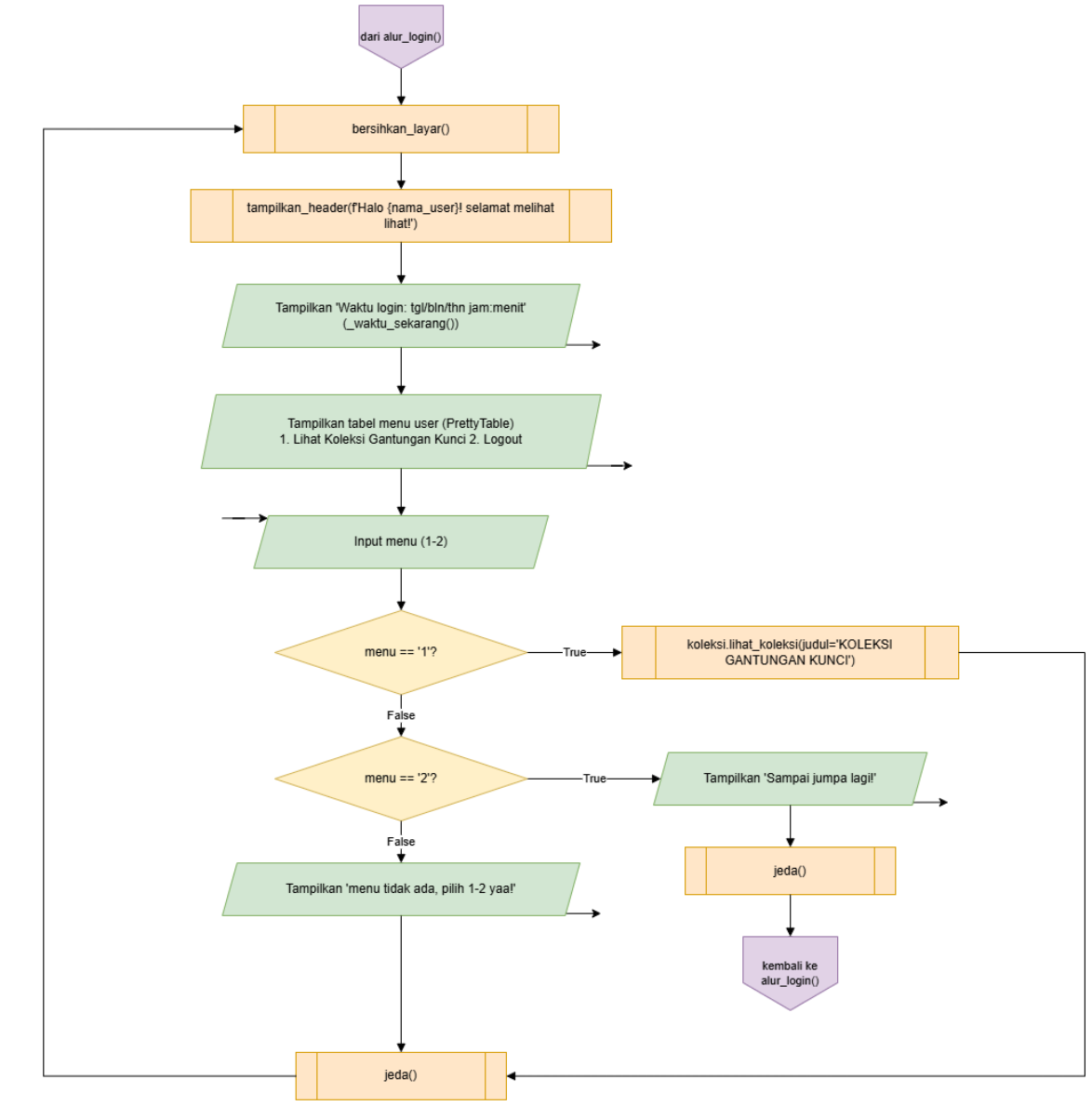


Gambar 1.7 Menu Admin

Menu ini berulang terus. Isinya: header, waktu login, tabel menu, lalu input 1-5.

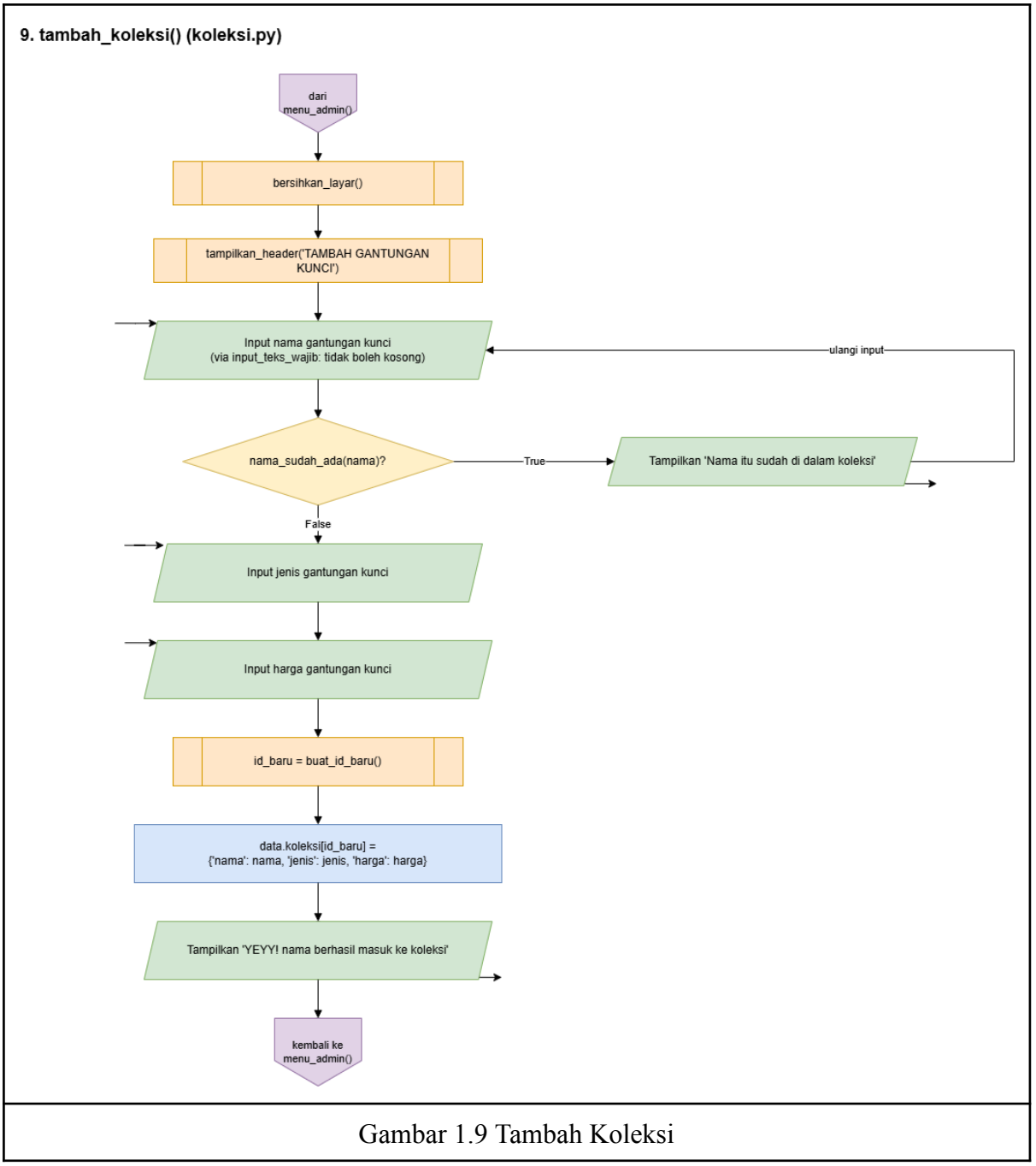
- Menu 1-4 memanggil fungsi tambah, lihat, ubah, dan hapus koleksi. Hasilnya bertemu di jeda() paling bawah, lalu kembali ke atas.
- Menu 5 menampilkan “Logout berhasil!”, jeda(), lalu kembali ke alur_login().
- Pilihan lain menampilkan pesan “Menu tidak ada”.

8. menu_user(nama_user) (menu.py)



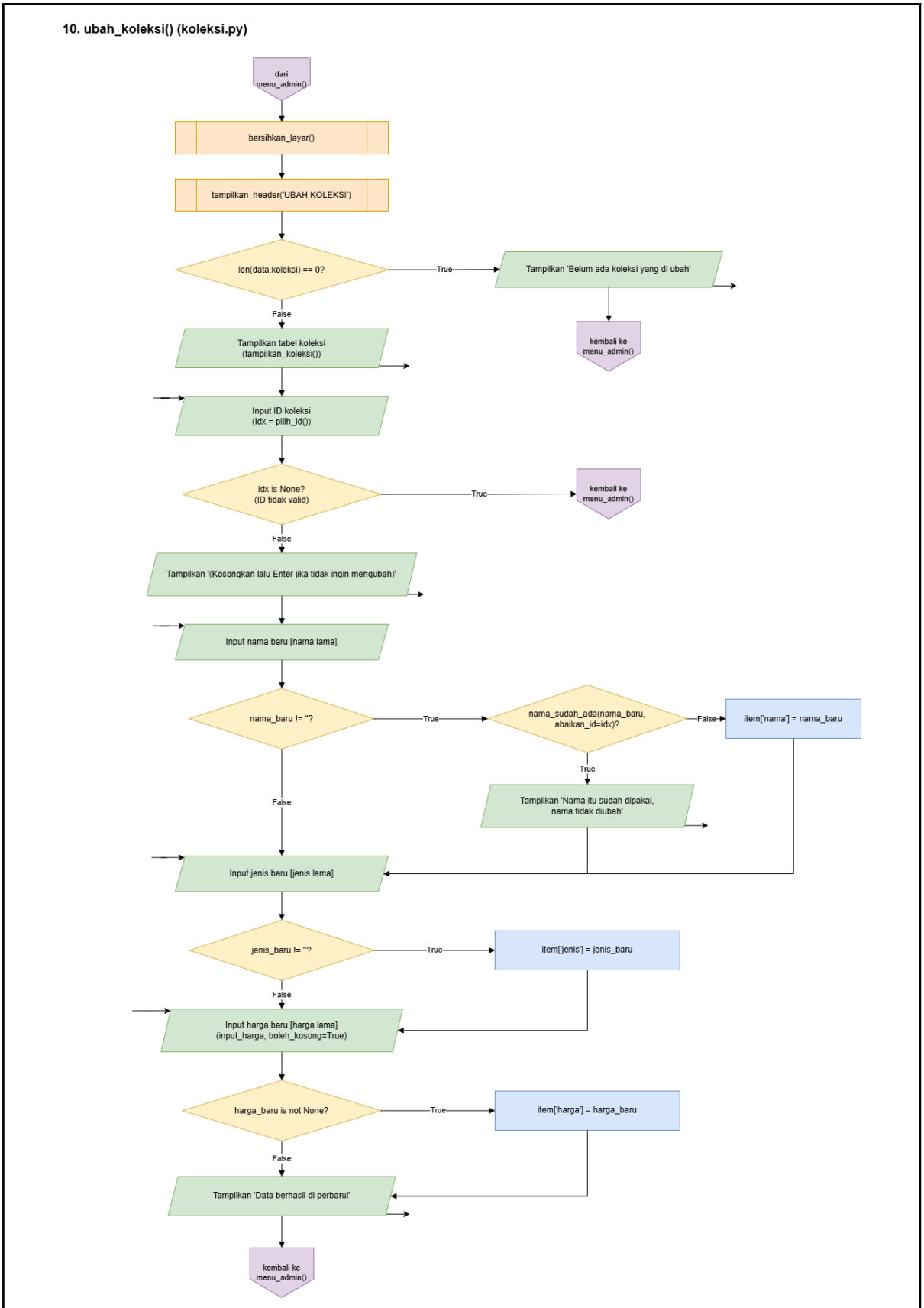
Gambar 1.8 Menu User

Sama seperti menu admin tetapi hanya 2 pilihan: 1 untuk melihat koleksi dan 2 untuk logout. User tidak bisa menambah, mengubah, atau menghapus data.



Gambar 1.9 Tambah Koleksi

User mengisi nama. Jika nama sudah ada di koleksi, pesan tampil dan input nama diulang (panah “ulangi input” di kanan). Setelah itu jenis dan harga diisi, ID baru dibuat lewat buat_id_baru(), data disimpan ke data.koleksi, dan pesan sukses tampil.

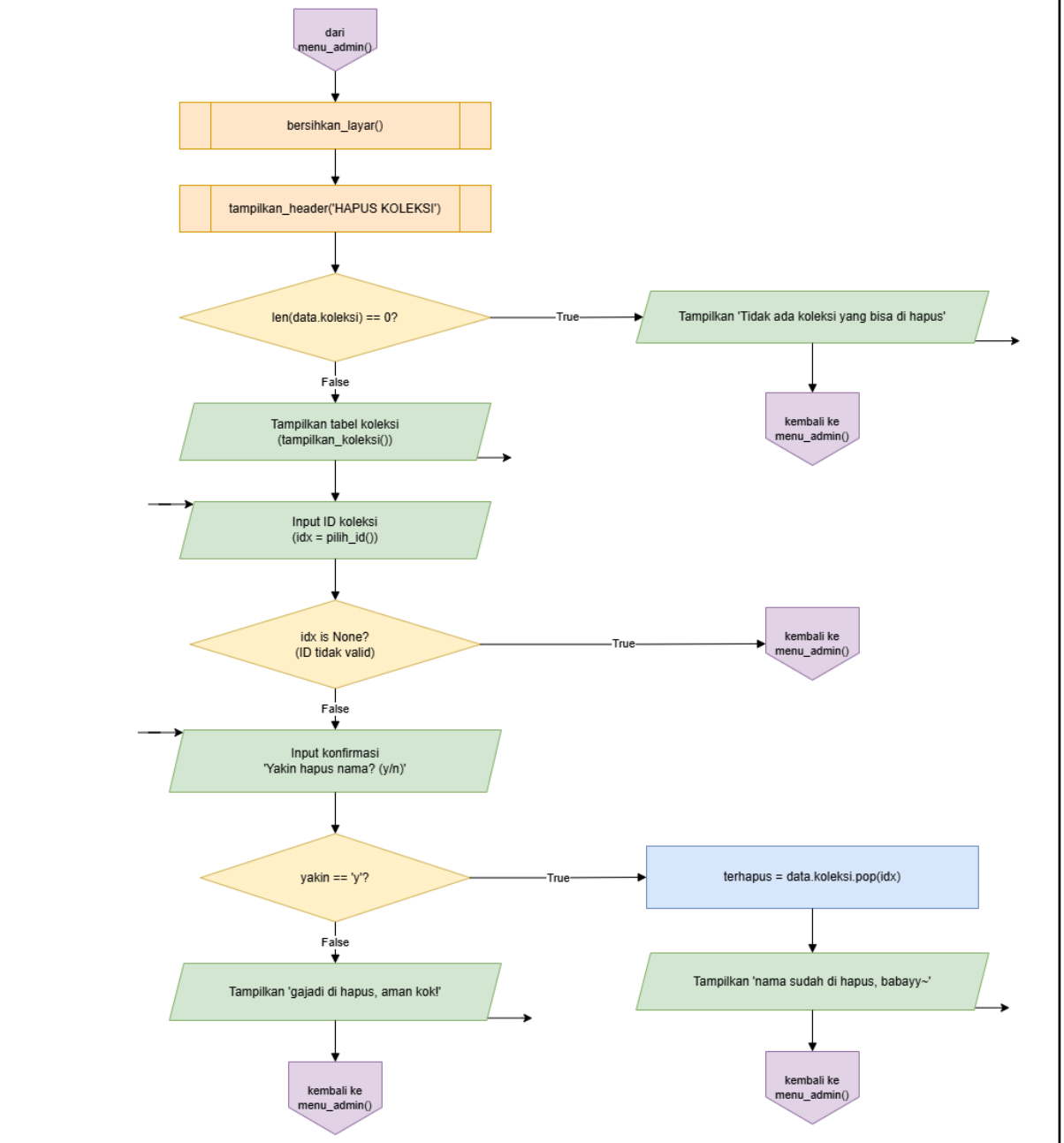


Gambar 1.10 Ubah Koleksi

- Jika koleksi kosong, pesan tampil dan fungsi selesai.

- Jika tidak kosong, tabel ditampilkan dan user memilih ID. Jika ID tidak valid (`None`), fungsi selesai.
- Berikutnya user mengisi nama, jenis, dan harga baru. Jika dikosongkan, data lama dipertahankan.
- Untuk nama, jika nama baru sudah dipakai item lain, nama tidak diubah.
- Pesan “Data berhasil di perbarui” tampil di akhir.

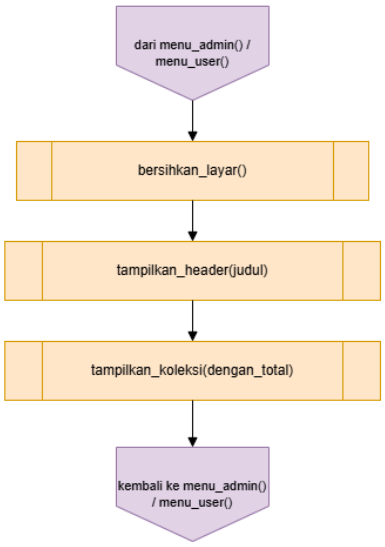
11. hapus_koleksi() (koleksi.py)



Gambar 1.11 Hapus koleksi

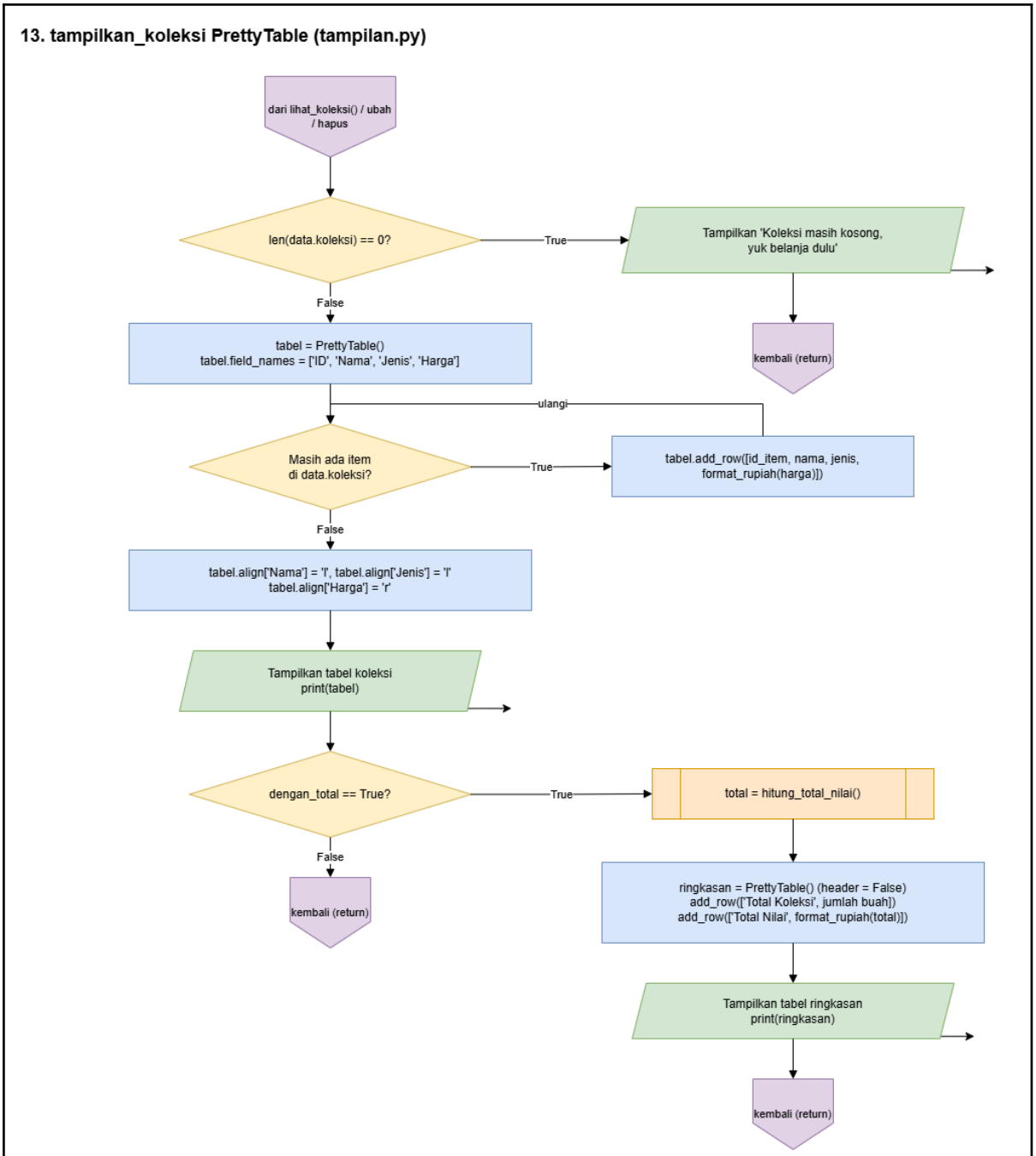
Alurnya: cek koleksi kosong, tampilkan tabel, pilih ID, lalu konfirmasi y/n. Jika y, data dihapus dengan pop(idx) dan pesan “sudah di hapus” tampil. Jika bukan y, pesan “gajadi di hapus” tampil.

12. lihat_koleksi (koleksi.py)



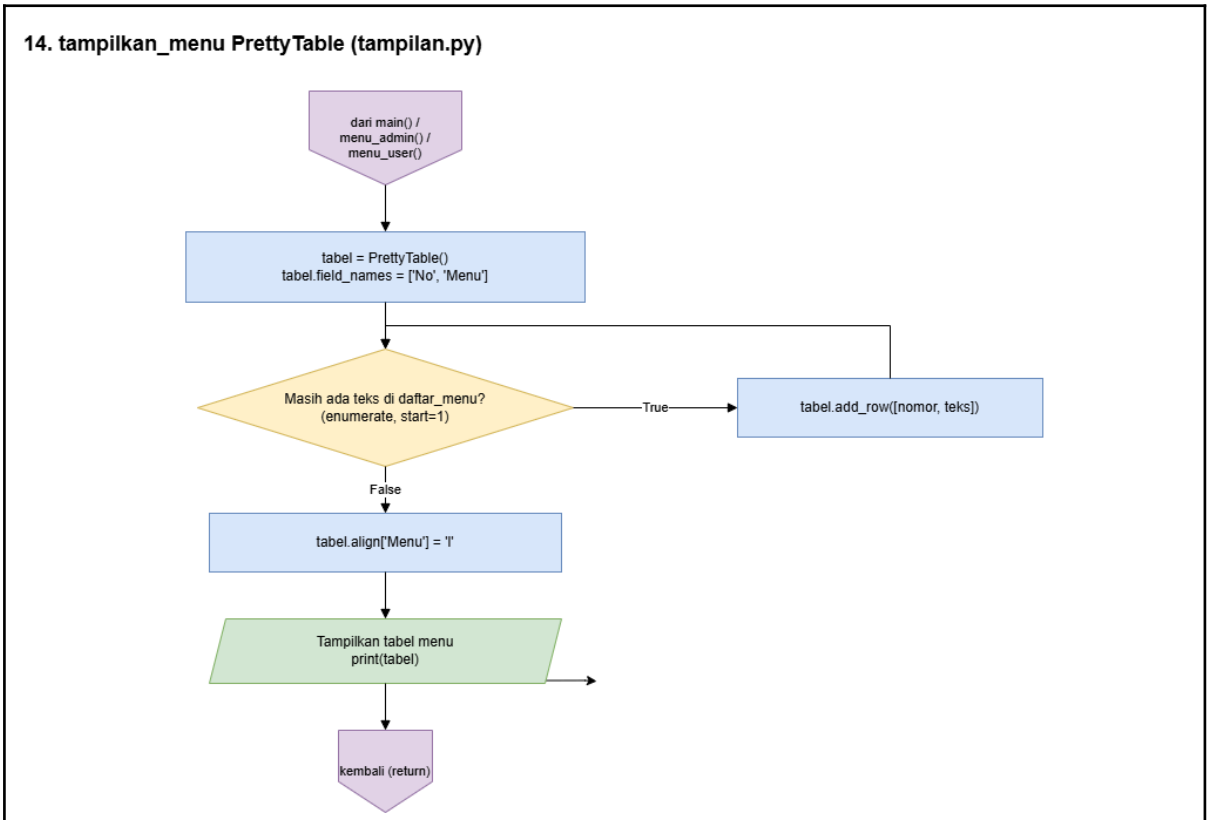
Gambar 1.12 lihat koleksi

Fungsi ini membersihkan layar, menampilkan header sesuai parameter judul, lalu memanggil tampilkan_koleksi(dengan_total). Admin memakai dengan_total=True sehingga ringkasan total ikut tampil. User melihat tabel saja.



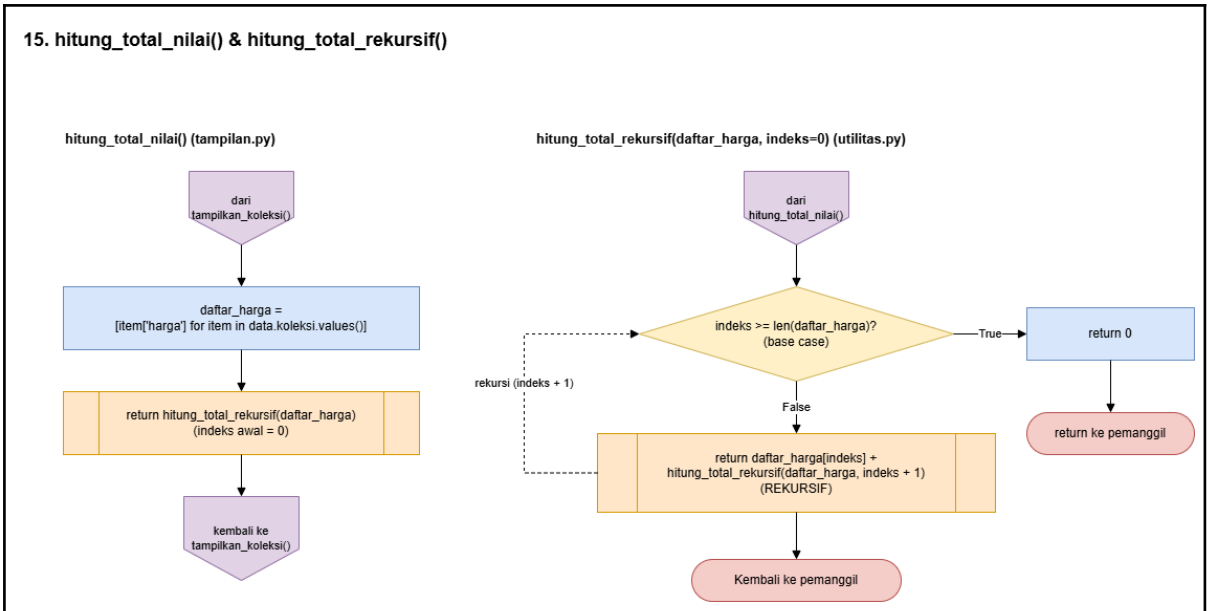
Gambar 1.13 Tampilkan koleksi

- Jika koleksi kosong, pesan “Koleksi masih kosong” tampil.
- Jika tidak kosong, tabel dibuat dengan kolom ID, Nama, Jenis, Harga. Setiap item ditambahkan dengan add_row (loop “ulangi”).
- Kolom Nama dan Jenis diratakan kiri, Harga kanan, lalu tabel dicetak.
- Jika dengan_total=True, total nilai dihitung dan tabel ringkasan kedua (Total Koleksi dan Total Nilai) dicetak.



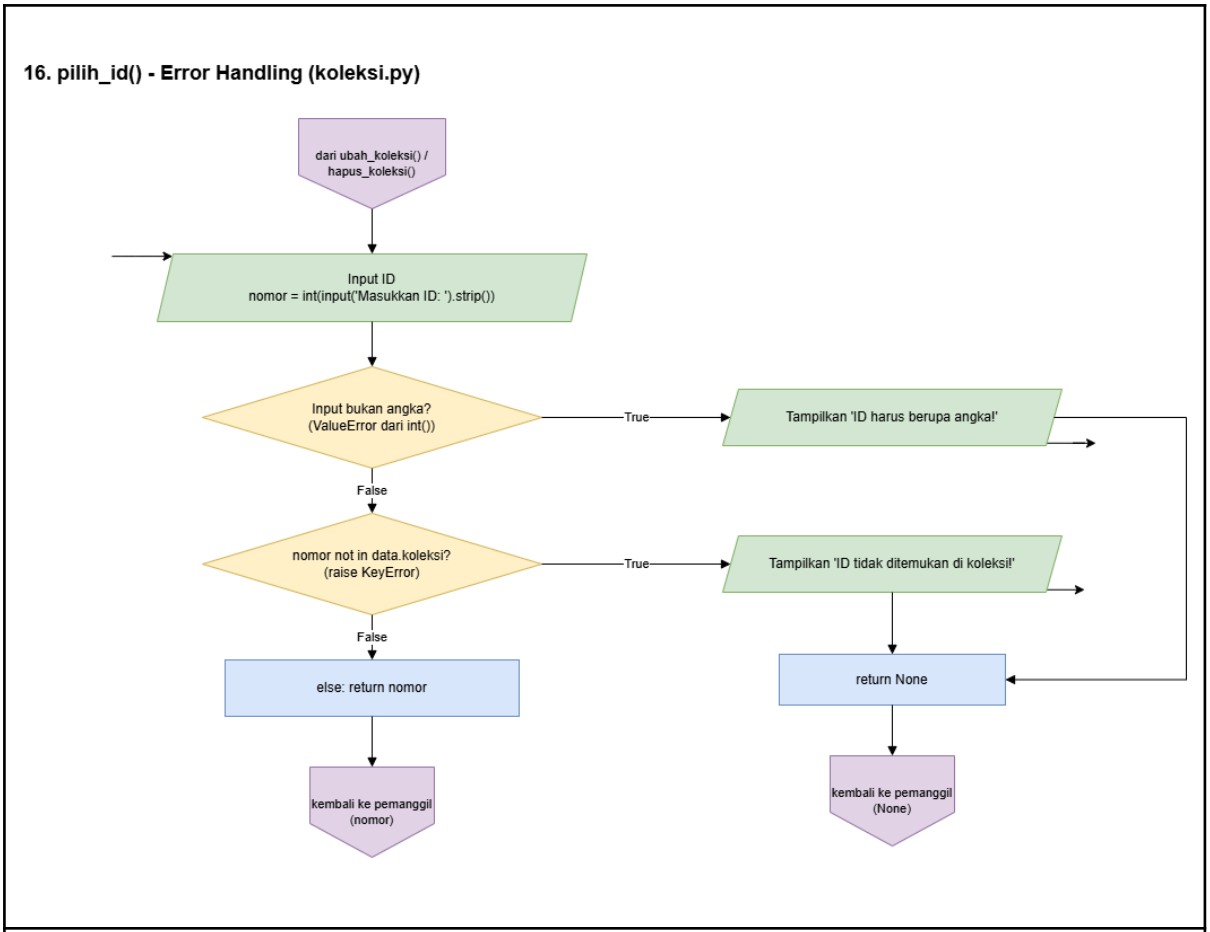
Gambar 1.14 Tampilkan menu

Fungsi ini membuat tabel dua kolom: No dan Menu. Setiap teks dalam daftar_menu ditambahkan dengan nomor mulai dari 1 (enumerate). Kolom Menu diratakan kiri, lalu tabel dicetak.



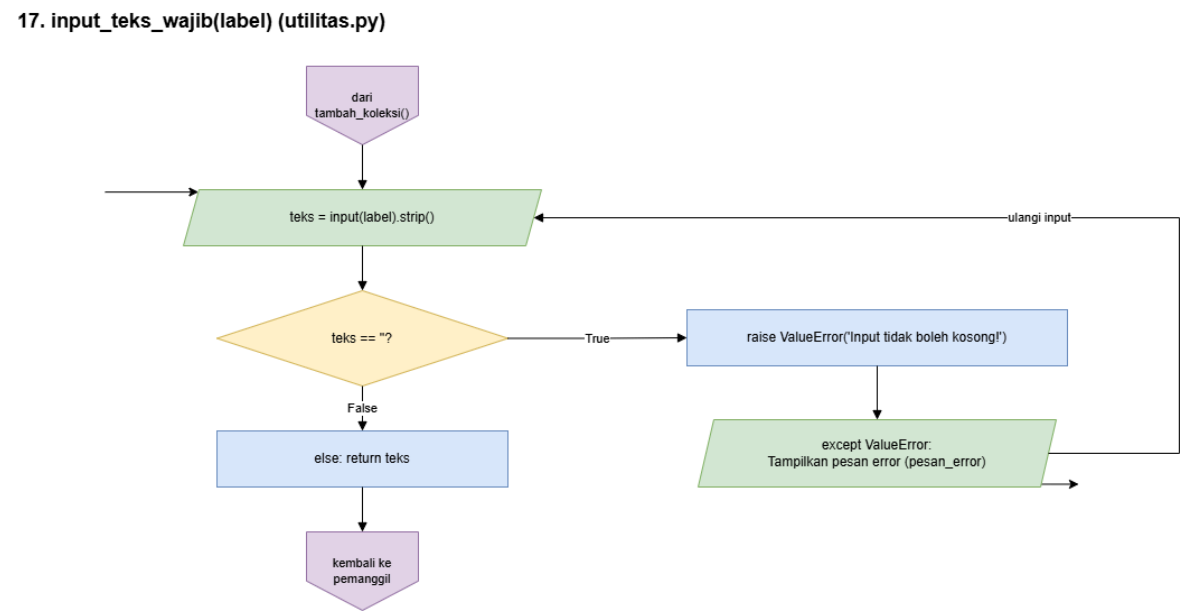
Gambar 1.15 Total nilai & Total rekursif

- hitung_total_nilai() mengumpulkan semua harga ke dalam list, lalu memanggil fungsi rekursif dengan indeks awal 0.
- Base case: jika indeks >= len(daftar_harga), hasilnya 0.
- Rekursi: selain itu, hasilnya harga[indeks] + hitung_total_rekursif(..., indeks + 1).
- Contoh dengan data awal: 15000 + (20000 + (12000 + 0)) = 47000.



Gambar 1.16 Pilih ID

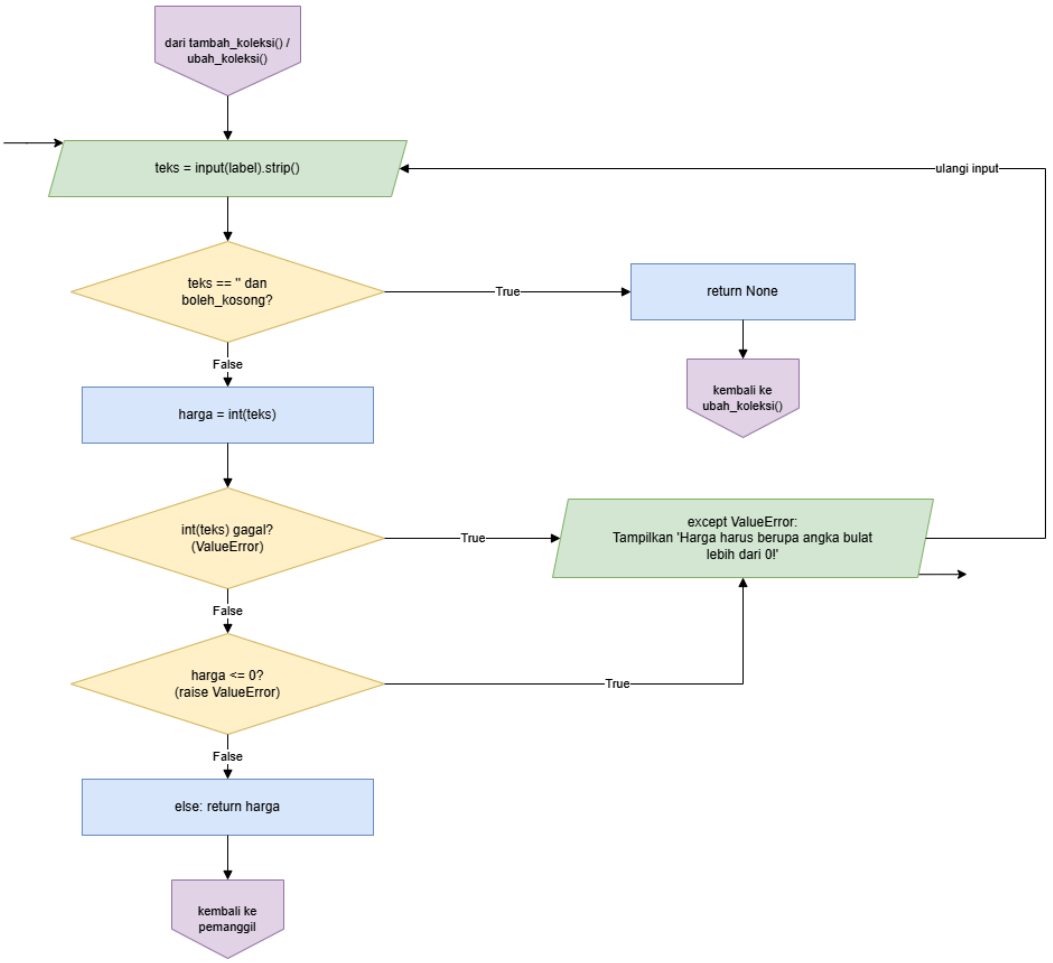
- User memasukkan ID.
- Jika bukan angka (ValueError), pesan “ID harus berupa angka!” tampil dan fungsi mengembalikan None.
- Jika angka tapi tidak ada di koleksi (KeyError), pesan “ID tidak ditemukan” tampil dan fungsi mengembalikan None.
- Jika valid, blok else mengembalikan nomor ID.



Gambar 1.17 input teks

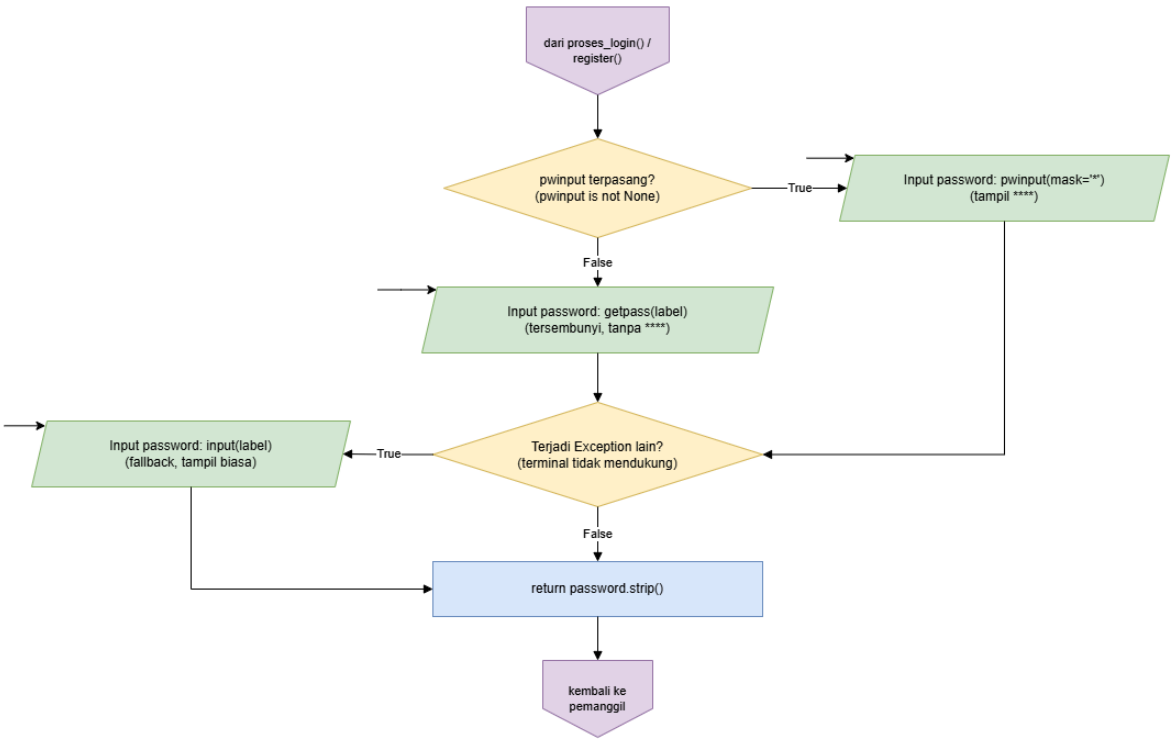
Fungsi ini meminta input dan membuang spasi di ujung teks. Jika kosong, ia melempar ValueError yang langsung ditangkap except untuk menampilkan pesan error, lalu input diulang. Jika terisi, teks dikembalikan.

18. input_harga(label, boleh_kosong=False) (utilitas.py)



Gambar 1.18 input harga

- Jika input kosong dan boleh_kosong=True, fungsi mengembalikan None. Ini dipakai saat ubah data supaya harga lama tidak berubah.
- Jika input bukan angka, atau angkanya ≤ 0, pesan “Harga harus berupa angka bulat lebih dari 0!” tampil dan input diulang.
- Jika valid, harga dikembalikan.



Gambar 1.19 baca password

Fungsi ini membaca password dengan tiga lapis cadangan:

- Jika pwinput terpasang, password tampil sebagai ****.
- Jika tidak, getpass dipakai dan ketikan disembunyikan total.
- Jika terminal tidak mendukung keduanya (terjadi exception), input() biasa dipakai sebagai fallback.

2. Deskripsi Singkat Program

Program “Koleksi Gantungan Kunci” dibuat untuk membantu pengelolaan data koleksi gantungan kunci secara sederhana dan terstruktur melalui tampilan menu di terminal. Dengan program ini, data koleksi tidak perlu dicatat secara manual karena dapat disimpan, dilihat, diubah, dan dihapus dengan mudah.

Fungsi utama program ini terbagi berdasarkan hak akses pengguna. Admin dapat menambah, melihat, mengubah, dan menghapus data koleksi, serta melihat jumlah koleksi dan total nilainya. Pengguna biasa hanya dapat melihat daftar koleksi tanpa bisa mengubah datanya. Pengguna baru juga dapat membuat akun sendiri melalui fitur register, dan setiap akun harus login terlebih dahulu sebelum bisa mengakses koleksi.

3. Source Code

A. main.py

```
    user_aktif, role_aktif = hasil
    if role_aktif == "Admin":
        menu.menu_admin()
    else:
        menu.menu_user(user_aktif)

def main():
    while True:
        bersihkan_layar()
        tampilkan_header("Koleksi Gantungan Kunci")
        tampilkan_menu(["Login", "Register", "Keluar"])
        pilihan = input("Pilih menu (1-3): ").strip()

        if pilihan == "1":
            alur_login()
        elif pilihan == "2":
            autentikasi.register()
            jeda()
        elif pilihan == "3":
```

```
            bersihkan_layar()
            pesan_info("Terima kasih sudah mampir! Jaga koleksimu ya")
            return
        else:
            pesan_error("Pilihan tidak valid, pilih 1-3 ya!")
            jeda()

if __name__ == "__main__":
    try:
        main()
    except KeyboardInterrupt:
        print("\nProgram dihentikan paksa (Ctrl+C). Sampai jumpa!")
    except EOFError:
        print("\nInput terputus. Program ditutup.")
    finally:
        print("Program selesai.")
```

Kode ini adalah titik masuk (main program) dari aplikasi konsol bernama “Koleksi Gantungan Kunci”. Program mengimpor modul-modul pendukung seperti autentikasi, menu, tampilan, dan utilitas, lalu menjalankan sebuah perulangan tak berujung di fungsi main() yang menampilkan menu utama berisi Login, Register, dan Keluar. Setiap iterasi membersihkan layar, menampilkan header, lalu meminta pengguna memilih angka 1 sampai 3.

Jika pengguna memilih Login, fungsi alur_login() dipanggil: program memanggil autentikasi.login(), dan bila hasilnya kosong (login gagal) akan muncul pesan error lalu kembali ke menu utama. Bila berhasil, fungsi mengembalikan nama pengguna dan perannya, kemudian program mengarahkan ke menu_admin() jika perannya Admin, atau ke menu_user() untuk pengguna biasa. Jika memilih Register, program memanggil fungsi pendaftaran akun lalu berhenti sejenak. Jika memilih Keluar, layar dibersihkan, muncul pesan perpisahan, dan program berhenti. Input selain 1-3 ditolak dengan pesan error.

Bagian paling bawah membungkus pemanggilan main() dengan penanganan error: tekan Ctrl+C atau input yang terputus (EOF) akan ditangkap sehingga program berhenti dengan pesan yang rapi, dan blok finally selalu mencetak “Program selesai” apa pun yang terjadi.

B. Data.py

```
MAKS_PERCOBAAN = 3

users = {
    "admin": {"password": "admin123", "role": "Admin"},
    "herul": {"password": "herul123", "role": "User"},
```

```
}

koleksi = {
    1: {"nama": "Si Kucing Hoki", "jenis": "Hewan", "harga": 15000},
    2: {"nama": "Dino Imut", "jenis": "Hewan", "harga": 20000},
    3: {"nama": "Sepatu Mini", "jenis": "Miniatur", "harga": 12000},
}
id_terakhir = 3
```

Kode ini berisi data awal (konfigurasi dan “database” sementara) untuk aplikasi koleksi gantungan kunci tadi. Semuanya disimpan di memori dalam bentuk variabel Python, sehingga data akan kembali ke kondisi awal setiap kali program dijalankan ulang.

MAKS_PERCOBAAN = 3 kemungkinan dipakai di fungsi login untuk membatasi berapa kali pengguna boleh salah memasukkan password sebelum dikembalikan ke menu utama. Dictionary users menyimpan dua akun bawaan, yaitu “admin” dengan peran Admin dan “herul” dengan peran User, masing-masing lengkap dengan password dalam bentuk teks biasa. Fungsi login() dan register() di modul autentikasi hampir pasti membaca dan menambah isi dictionary ini.

Dictionary koleksi menyimpan tiga item gantungan kunci awal, dengan angka ID sebagai kunci dan data nama, jenis (Hewan atau Miniatur), serta harga sebagai isinya. Variabel id_terakhir = 3 mencatat ID terbesar yang sudah terpakai, jadi saat ada item baru ditambahkan, program tinggal menaikkannya menjadi 4 dan seterusnya agar ID tidak bentrok.

C. Utilitas.py

```
import os

from colorama import Fore, Style, init
init(autoreset=True)

CLEAR_CMD = "cls" if os.name == "nt" else "clear"

def bersihkan_layar():
    os.system(CLEAR_CMD)

def tampilkan_header(judul, lebar=45):
    print(Fore.CYAN + "=" * lebar)
    print(Fore.CYAN + Style.BRIGHT + f"  {judul}")
    print(Fore.CYAN + "=" * lebar)

def jeda():
```

```
    try:
        input(Style.DIM + "Tekan Enter untuk lanjut...")
    except EOFError:
        pass

def pesan_sukses(teks):
    print(Fore.GREEN + teks)

def pesan_error(teks):
    print(Fore.RED + teks)

def pesan_info(teks):
    print(Fore.YELLOW + teks)

def format_rupiah(angka):
    return "Rp" + f"{angka:,}".replace(",", ".")

def input_teks_wajib(label):
    while True:
        try:
            teks = input(label).strip()
            if teks == "":
                raise ValueError("Input tidak boleh kosong!")
        except ValueError as e:
            pesan_error(str(e))
        else:
            return teks

def input_harga(label, boleh_kosong=False):
    while True:
        try:
            teks = input(label).strip()
            if teks == "" and boleh_kosong:
                return None
            harga = int(teks)
            if harga <= 0:
                raise ValueError
        except ValueError:
            pesan_error("Harga harus berupa angka bulat lebih dari 0!")
        else:
            return harga

def hitung_total_rekursif(daftar_harga, indeks=0):
    if indeks >= len(daftar_harga):
        return 0
    return daftar_harga[indeks] + hitung_total_rekursif(daftar_harga, indeks
+ 1)
```

Ini adalah modul utilitas.py, kumpulan fungsi bantu yang dipakai di seluruh aplikasi, terutama untuk tampilan terminal dan validasi input. Modul ini memakai library colorama agar teks berwarna bisa tampil di terminal Windows maupun lainnya, dan init(autoreset=True) membuat warna otomatis kembali normal setelah setiap print.

Untuk urusan tampilan, bersihkan_layar() menjalankan perintah cls di Windows atau clear di sistem lain (ditentukan oleh CLEAR_CMD), sedangkan tampilkan_header() mencetak judul tebal berwarna cyan yang diapit garis = dengan lebar default 45 karakter. Fungsi jeda() menunggu pengguna menekan Enter dan mengabaikan error bila input terputus. Tiga fungsi pesan, yaitu pesan_sukses, pesan_error, dan pesan_info, hanya mencetak teks dengan warna hijau, merah, dan kuning agar status operasi mudah dibedakan. Fungsi format_rupiah() mengubah angka seperti 15000 menjadi "Rp15.000" dengan trik mengganti pemisah ribuan koma menjadi titik.

Bagian validasi ada di dua fungsi input. input_teks_wajib() terus mengulang pertanyaan sampai pengguna mengisi sesuatu yang tidak kosong, memanfaatkan pola try/except/else di mana error ditangkap dan ditampilkan, sedangkan else mengembalikan hasil bila input valid. input_harga() bekerja serupa tetapi memastikan input berupa bilangan bulat lebih dari 0, dan dengan boleh_kosong=True ia mengembalikan None bila pengguna hanya menekan Enter, yang biasanya berguna saat mengedit data dan ingin mempertahankan harga lama.

Terakhir, hitung_total_rekursif() menjumlahkan seluruh isi daftar harga secara rekursif: ia mengambil harga pada indeks tertentu lalu menambahkannya dengan hasil pemanggilan dirinya sendiri untuk indeks berikutnya, dan berhenti dengan nilai 0 ketika indeks sudah melewati akhir daftar. Hasilnya sama seperti sum(), tetapi ditulis rekursif, kemungkinan untuk memenuhi ketentuan tugas.

D. Tampilan py

```
from prettytable import PrettyTable

import data
from utilitas import format_rupiah, hitung_total_rekursif, pesan_info

def hitung_total_nilai():
    daftar_harga = [item["harga"] for item in data.koleksi.values()]
    return hitung_total_rekursif(daftar_harga)

def tampilkan_menu(daftar_menu):
    tabel = PrettyTable()
    tabel.field_names = ["No", "Menu"]
    for nomor, teks in enumerate(daftar_menu, start=1):
        tabel.add_row([nomor, teks])
    tabel.align["Menu"] = "l"
```

```
    print(tabel)

def tampilkan_koleksi(dengan_total=False):
    if len(data.koleksi) == 0:
        pesan_info("Koleksi masih kosong, yuk belanja dulu")
        return

    tabel = PrettyTable()
    tabel.field_names = ["ID", "Nama", "Jenis", "Harga"]
    for id_item, item in data.koleksi.items():
        tabel.add_row([id_item, item["nama"], item["jenis"],
format_rupiah(item["harga"])])
    tabel.align["Nama"] = "l"
    tabel.align["Jenis"] = "l"
    tabel.align["Harga"] = "r"
    print(tabel)

    if dengan_total:
        ringkasan = PrettyTable()
        ringkasan.header = False
        ringkasan.add_row(["Total Koleksi", f"{len(data.koleksi)} buah"])
        ringkasan.add_row(["Total Nilai",
format_rupiah(hitung_total_nilai())])
        ringkasan.align = "l"
        print(ringkasan)
```

Ini adalah modul tampilan.py, yang bertugas mencetak data dalam bentuk tabel rapi di terminal memakai library prettytable. Modul ini mengimpor data untuk membaca isi koleksi, serta beberapa fungsi bantu dari utilitas seperti format_rupiah, hitung_total_rekursif, dan pesan_info.

Fungsi hitung_total_nilai() mengambil seluruh harga dari dictionary data.koleksi lewat list comprehension, lalu menyerahkannya ke hitung_total_rekursif() dari modul utilitas untuk dijumlahkan. Inilah tempat fungsi rekursif tadi benar-benar dipakai.

Fungsi tampilkan_menu() menerima sebuah daftar teks, misalnya ["Login", "Register", "Keluar"], lalu membuat tabel dua kolom berisi nomor dan nama menu. Nomor dibuat otomatis dengan enumerate yang dimulai dari 1, dan kolom Menu diratakan ke kiri. Fungsi inilah yang dipanggil di main() pada file pertama tadi.

Fungsi tampilkan_koleksi() menampilkan seluruh gantungan kunci dalam tabel berkolom ID, Nama, Jenis, dan Harga. Bila koleksi kosong, ia hanya menampilkan pesan info berwarna kuning lalu berhenti. Kolom Nama dan Jenis rata kiri, sedangkan Harga rata kanan dan sudah diformat menjadi rupiah, sehingga angka mudah dibandingkan. Jika parameter dengan_total=True, fungsi mencetak tabel ringkasan tambahan tanpa header yang berisi jumlah item dan total nilai seluruh koleksi, sehingga tampilan biasa dan tampilan lengkap bisa dipilih sesuai kebutuhan pemanggilnya.

E. Koleksi py

```
import data
from tampilan import tampilkan_koleksi
from utilitas import (
    bersihkan_layar,
    input_harga,
    input_teks_wajib,
    pesan_error,
    pesan_info,
    pesan_sukses,
    tampilkan_header,
)

def buat_id_baru():
    data.id_terakhir += 1
    return data.id_terakhir


def nama_sudah_ada(nama, abaikan_id=None):
    for id_item, item in data.koleksi.items():
        if id_item != abaikan_id and item["nama"].lower() == nama.lower():
            return True
    return False

def pilih_id():
    try:
        nomor = int(input("Masukkan ID: ").strip())
        if nomor not in data.koleksi:
            raise KeyError(nomor)
    except ValueError:
        pesan_error("ID harus berupa angka!")
    except KeyError:
        pesan_error("ID tidak ditemukan di koleksi!")
    else:
        return nomor
    return None

def lihat_koleksi(dengan_total=False, judul="DAFTAR KOLEKSI"):
    bersihkan_layar()
    tampilkan_header(judul)
    tampilkan_koleksi(dengan_total=dengan_total)

def tambah_koleksi():
```

```
    bersihkan_layar()
    tampilkan_header("TAMBAH GANTUNGAN KUNCI")

    while True:
        nama = input_teks_wajib("Nama gantungan kunci: ")
        if nama_sudah_ada(nama):
            pesan_error("Nama itu sudah di dalam koleksi")
        else:
            break
    jenis = input_teks_wajib("Jenis gantungan kunci: ")
    harga = input_harga("Harga gantungan kunci: ")

    id_baru = buat_id_baru()
    data.koleksi[id_baru] = {"nama": nama, "jenis": jenis, "harga": harga}
    pesan_sukses(f"YEYY! '{nama}' berhasil masuk ke koleksi")

def ubah_koleksi():
    bersihkan_layar()
    tampilkan_header("UBAH KOLEKSI")
    if len(data.koleksi) == 0:
        pesan_info("Belum ada koleksi yang di ubah")
        return
    tampilkan_koleksi()
    idx = pilih_id()
    if idx is None:
        return

    item = data.koleksi[idx]
    print("(Kosongkan lalu Enter jika tidak ingin mengubah)")
    nama_baru = input(f"Nama baru  [{item['nama']}]: ").strip()
    if nama_baru != "":
        if nama_sudah_ada(nama_baru, abaikan_id=idx):
            pesan_error("Nama itu sudah dipakai, nama tidak diubah")
        else:
            item["nama"] = nama_baru

    jenis_baru = input(f"Jenis baru [{item['jenis']}]: ").strip()
    if jenis_baru != "":
        item["jenis"] = jenis_baru

    harga_baru = input_harga(f"Harga baru [{item['harga']}]: ",
boleh_kosong=True)
    if harga_baru is not None:
        item["harga"] = harga_baru
    pesan_sukses("Data berhasil di perbarui")

def hapus_koleksi():
    bersihkan_layar()
```

```
    tampilkan_header("HAPUS KOLEKSI")
    if len(data.koleksi) == 0:
        pesan_info("Tidak ada koleksi yang bisa di hapus")
        return
    tampilkan_koleksi()
    idx = pilih_id()
    if idx is None:
        return
    yakin = input(f"Yakin hapus '{data.koleksi[idx]['nama']}'? (y/n):
").strip().lower()
    if yakin == "y":
        terhapus = data.koleksi.pop(idx)
        pesan_sukses(f"'{terhapus['nama']}' sudah di hapus, babayy~")
    else:
        pesan_info("gajadi di hapus, aman kok!")
```

Ini adalah modul koleksi.py, yang menangani operasi CRUD (tambah, lihat, ubah, hapus) pada data gantungan kunci di data.koleksi. Modul ini memadukan data dari data, tampilan tabel dari tampilan, serta fungsi input dan pesan dari utilitas.

Ada tiga fungsi bantu di bagian atas. buat_id_baru() menaikkan data.id_terakhir satu angka lalu mengembalikannya, sehingga ID baru selalu unik dan tidak dipakai ulang meski ada item yang dihapus. nama_sudah_ada() memeriksa apakah sebuah nama sudah dipakai item lain tanpa membedakan huruf besar kecil, dan parameter abaikan_id memungkinkan item yang sedang diedit tidak dianggap bentrok dengan namanya sendiri. pilih_id() meminta pengguna mengetik ID, lalu memvalidasinya dengan penanganan error: ValueError bila bukan angka, dan KeyError (dilempar manual dengan raise) bila ID tidak ada di koleksi. Fungsi ini mengembalikan nomor ID bila valid atau None bila gagal.

Fungsi lihat_koleksi() membersihkan layar, menampilkan header, lalu memanggil tampilkan_koleksi(), dengan opsi total yang bisa dinyalakan sesuai kebutuhan, misalnya untuk tampilan admin. Fungsi tambah_koleksi() meminta nama (diulang terus sampai tidak duplikat), jenis, dan harga, kemudian menyimpan item baru dengan ID dari buat_id_baru() dan menampilkan pesan sukses.

Fungsi ubah_koleksi() lebih rumit: setelah menampilkan tabel dan memilih ID, pengguna diminta mengisi nama, jenis, dan harga baru, tetapi setiap kolom boleh dikosongkan agar nilai lama dipertahankan. Nama baru tetap dicek agar tidak bentrok dengan item lain. Terakhir, hapus_koleksi() menampilkan tabel, meminta ID, lalu meminta konfirmasi “y/n” sebelum benar-benar menghapus item dengan pop(). Jika koleksi kosong, kedua fungsi ubah dan hapus langsung berhenti dengan pesan info.

F. Autentikasi py

```
from getpass import getpass

try:
    import pwinput
except ImportError:
    pwinput = None

import data
from utilitas import (
    bersihkan_layar,
    pesan_error,
    pesan_sukses,
    tampilkan_header,
)

def _baca_password(label):
    try:
        if pwinput is not None:
            return pwinput.pwinput(prompt=label, mask="*").strip()
        return getpass(label).strip()
    except (EOFError, KeyboardInterrupt):
        raise
    except Exception:
        return input(label).strip()

def proses_login(sisa):
    if sisa == 0:
        return None
    username = input("Username: ").strip()
    password = _baca_password("Password: ")
    akun = data.users.get(username)
    if akun is not None and akun["password"] == password:
        return username, akun["role"]
    pesan_error(f"Username atau Password anda salah! Sisa percobaan anda:
{sisa - 1}")
    return proses_login(sisa - 1)

def login():
    bersihkan_layar()
    tampilkan_header("LOGIN")
    return proses_login(data.MAKS_PERCOBAAN)

def register():
    bersihkan_layar()
    tampilkan_header("REGISTER AKUN BARU")
    username_baru = input("Username baru: ").strip()
```

```
    sudah_dipakai = any(u.lower() == username_baru.lower() for u in
data.users)
    if username_baru == "":
        pesan_error("username tidak boleh kosong!")
    elif " " in username_baru:
        pesan_error("username tidak boleh mengandung spasi!")
    elif sudah_dipakai:
        pesan_error("Username telah di pakai, cari yang lain ya!")
    else:
        password_baru = _baca_password("Password baru: ")
        if len(password_baru) < 4:
            pesan_error("Password minimal 4 karakter!")
        else:
            data.users[username_baru] = {"password": password_baru, "role":
"User"}
            pesan_sukses(f"Akun '{username_baru}' berhasil dibuat! silahkan
login")
```

Ini adalah modul autentikasi.py, yang mengurus login dan pendaftaran akun dengan membaca serta menulis ke data.users.

Fungsi _baca_password() membaca password secara tersembunyi: memakai pwinput (menampilkan bintang *) bila library-nya terpasang, kalau tidak memakai getpass bawaan Python yang tidak menampilkan apa pun. Jika keduanya bermasalah di terminal tertentu, ia jatuh kembali ke input() biasa, sedangkan Ctrl+C dan EOF sengaja dilempar ulang agar ditangani di main().

Fungsi login() membersihkan layar, menampilkan header, lalu memanggil proses_login() dengan jatah percobaan dari data.MAKS_PERCOBAAN (3 kali). proses_login() bekerja secara rekursif: ia meminta username dan password, dan bila cocok dengan data akun, langsung mengembalikan pasangan username dan role. Bila salah, ia menampilkan sisa percobaan lalu memanggil dirinya sendiri dengan jatah berkurang satu, dan ketika jatah habis ia mengembalikan None, yang oleh alur_login() di file utama dianggap login gagal.

Fungsi register() meminta username baru lalu memvalidasinya: tidak boleh kosong, tidak boleh mengandung spasi, dan tidak boleh sama dengan username lain tanpa membedakan huruf besar kecil. Jika lolos, program meminta password minimal 4 karakter, kemudian menyimpan akun baru dengan role “User”. Dengan begitu, hanya akun admin bawaan yang bisa menjadi Admin, sementara semua pendaftar baru otomatis pengguna biasa.

G. Menu py

```
from datetime import datetime

import koleksi
from tampilan import tampilkan_menu
from utilitas import bersihkan_layar, jeda, pesan_error, pesan_info,
tampilkan_header

def _waktu_sekarang():
    return datetime.now().strftime("%d/%m/%Y %H:%M")

def menu_admin():
    while True:
        bersihkan_layar()
        tampilkan_header("Halo admin! Mau ngapain nih hari ini")
        pesan_info(f"Waktu login: {_waktu_sekarang()}")
        tampilkan_menu([
            "Tambahkan Gantungan Kunci",
            "Lihat Semua Koleksi",
            "Ubah Koleksi",
            "Hapus Koleksi",
            "Logout",
        ])
        pilihan = input("Pilih menu (1-5): ").strip()

        if pilihan == "1":
            koleksi.tambah_koleksi()
        elif pilihan == "2":
            koleksi.lihat_koleksi(dengan_total=True)
        elif pilihan == "3":
            koleksi.ubah_koleksi()
        elif pilihan == "4":
            koleksi.hapus_koleksi()
        elif pilihan == "5":
            pesan_info("Logout berhasil!")
            jeda()
            return
        else:
            pesan_error("Menu tidak ada, pilih 1-5 yaa!")
        jeda()

def menu_user(nama_user):
    while True:
        bersihkan_layar()
        tampilkan_header(f"Halo {nama_user}! selamat melihat lihat!")
        pesan_info(f"Waktu login: {_waktu_sekarang()}")
        tampilkan_menu([
```

```
            "Lihat Koleksi Gantungan Kunci",
            "Logout",
        ])
        pilihan = input("Pilih menu (1-2): ").strip()

        if pilihan == "1":
            koleksi.lihat_koleksi(judul="KOLEKSI GANTUNGAN KUNCI")
        elif pilihan == "2":
            pesan_info("Sampai jumpa lagi!")
            jeda()
            return
        else:
            pesan_error("menu tidak ada, pilih 1-2 yaa!")
        jeda()
```

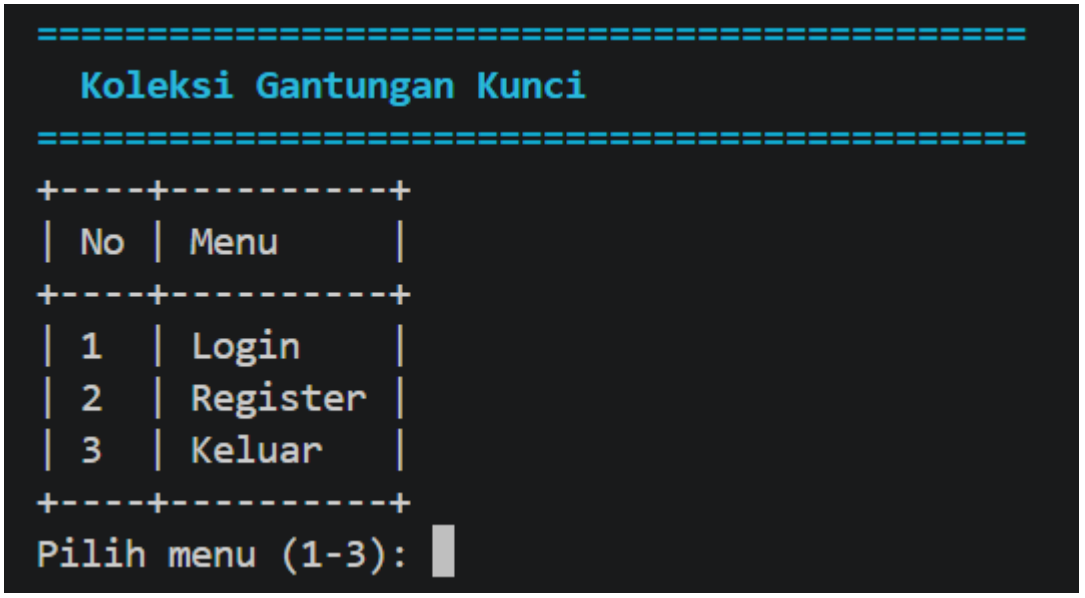
Ini adalah modul menu.py, yang berisi dua menu utama setelah login berhasil: satu untuk Admin dan satu untuk User biasa. Keduanya memanggil fungsi dari modul koleksi dan memakai tampilkan_menu() untuk mencetak daftar pilihan dalam bentuk tabel.

Fungsi _waktu_sekarang() hanya mengambil waktu saat ini dan memformatnya menjadi “hari/bulan/tahun jam:menit”. Hasilnya ditampilkan sebagai info “Waktu login” di kedua menu.

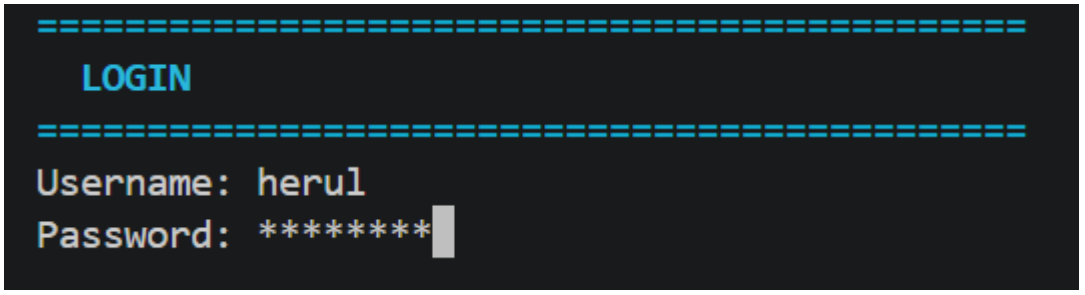
menu_admin() berjalan dalam perulangan tak berujung yang menampilkan lima pilihan: tambah, lihat, ubah, hapus koleksi, dan logout. Setiap pilihan langsung diteruskan ke fungsi yang sesuai di modul koleksi. Khusus untuk melihat koleksi, admin memanggilnya dengan dengan_total=True sehingga muncul ringkasan jumlah item dan total nilai. Pilihan logout menampilkan pesan lalu keluar dari fungsi (return), yang membawa program kembali ke menu utama di file pertama. Input yang tidak valid memunculkan pesan error.

menu_user(nama_user) jauh lebih sederhana karena pengguna biasa hanya punya dua pilihan: melihat koleksi (tanpa total nilai dan dengan judul “KOLEKSI GANTUNGAN KUNCI”) dan logout. Nama pengguna dipakai untuk menyapa di header. Dengan begitu pembagian hak akses terlihat jelas: hanya admin yang boleh mengubah data, sedangkan user hanya bisa melihat.

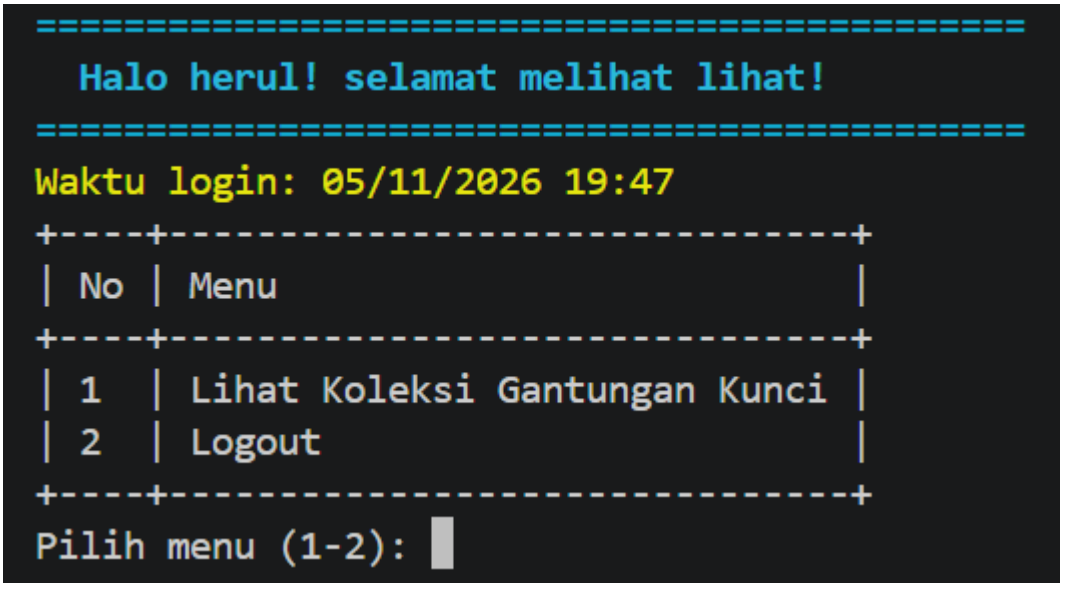
4. Hasil Output



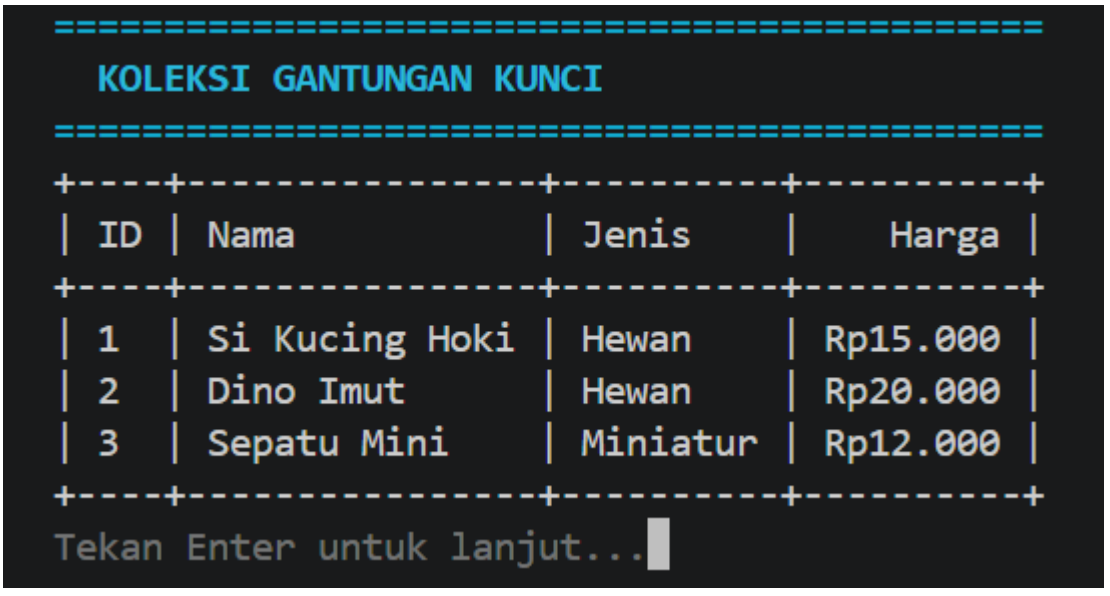
4.1 Ouput Menu Utama



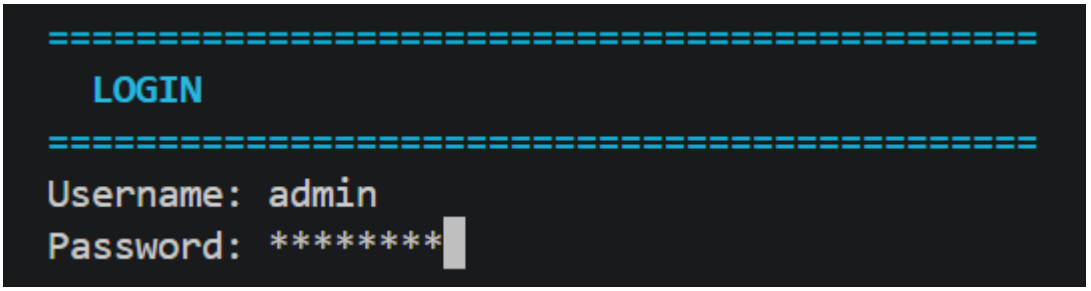
4.2 Ouput Menu Login (User)



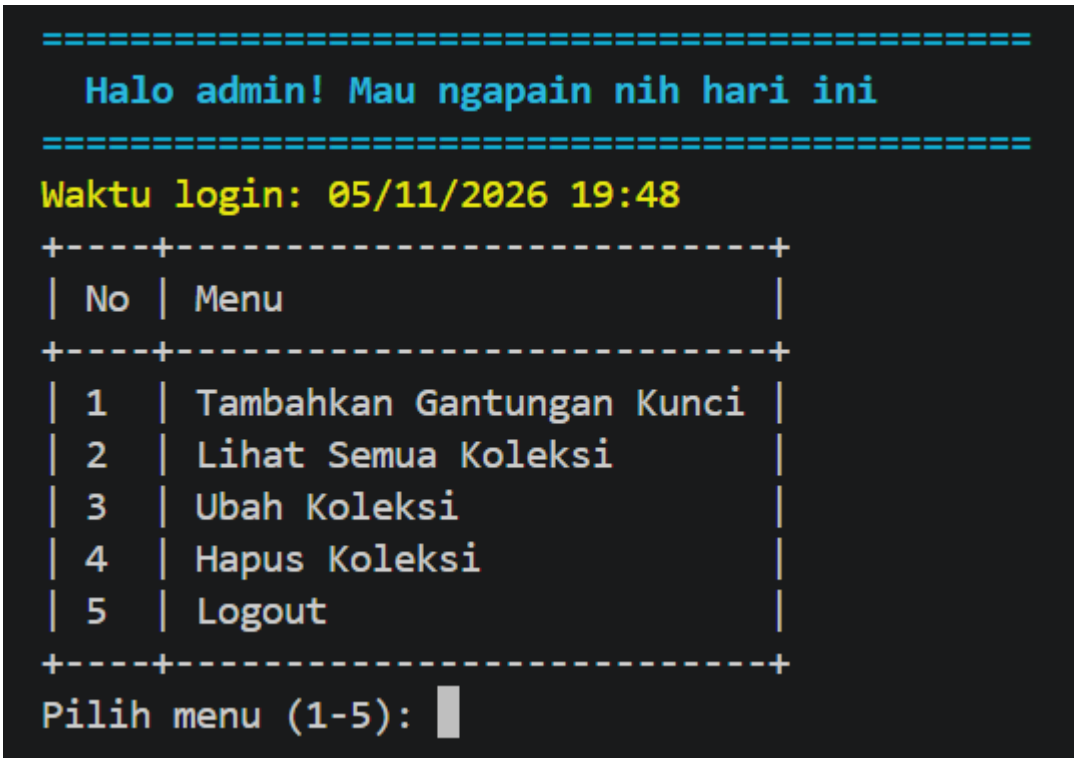
4.3 Output Menu User



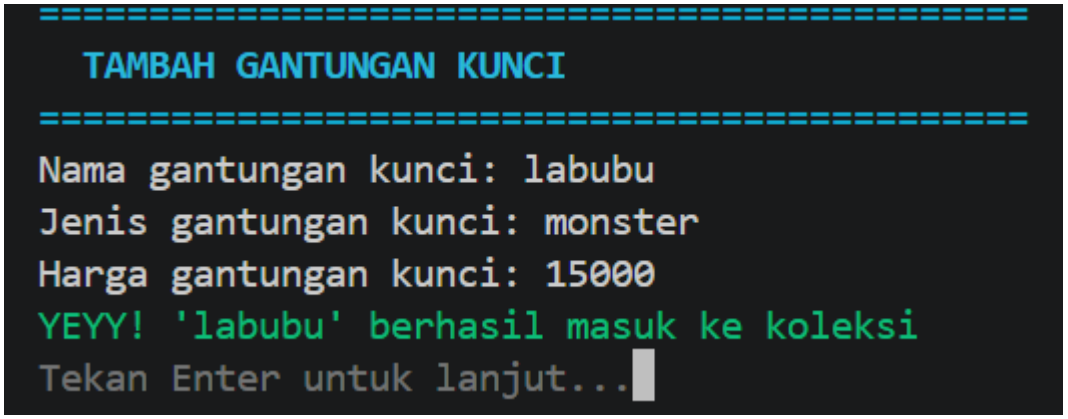
4.4 Output Menu Tampilan Koleksi



4.5 Output Menu Login Admin



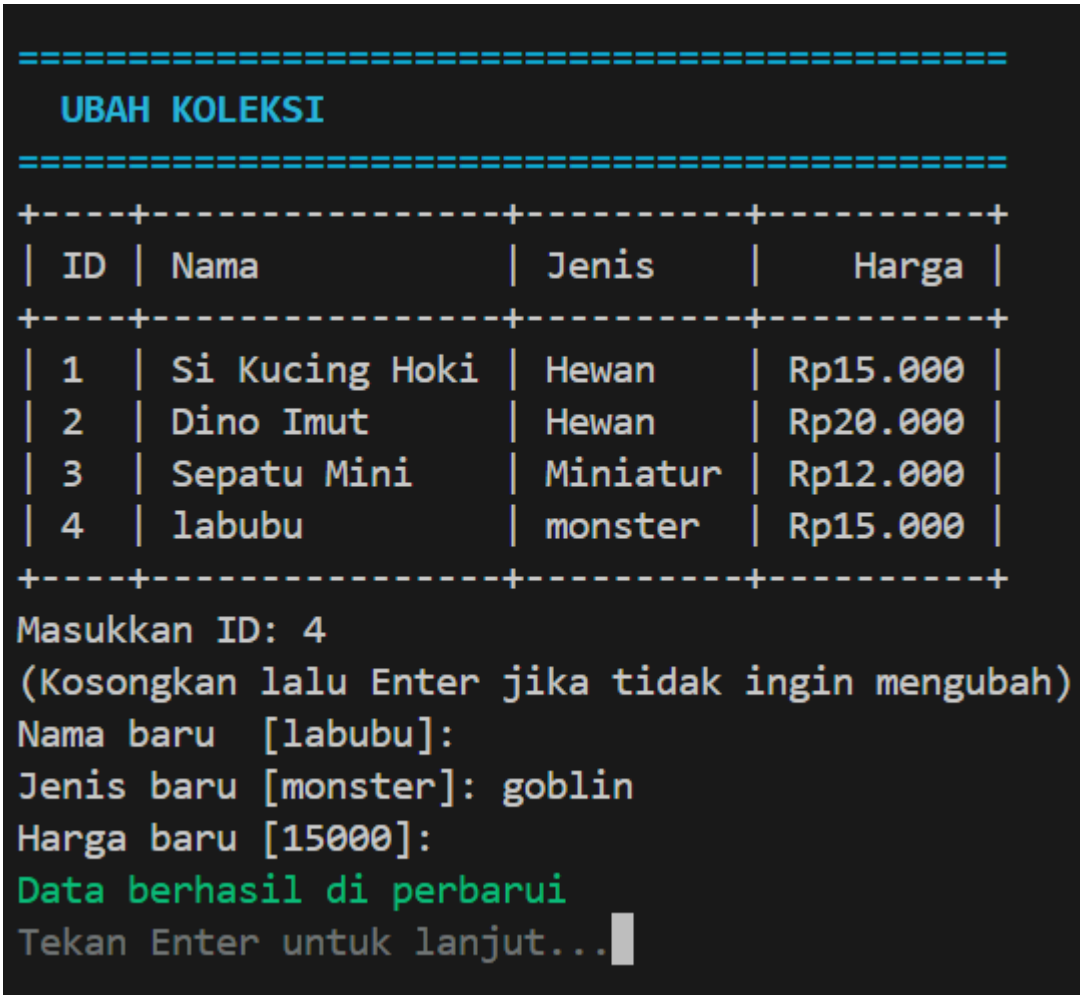
4.6 Output Menu Admin



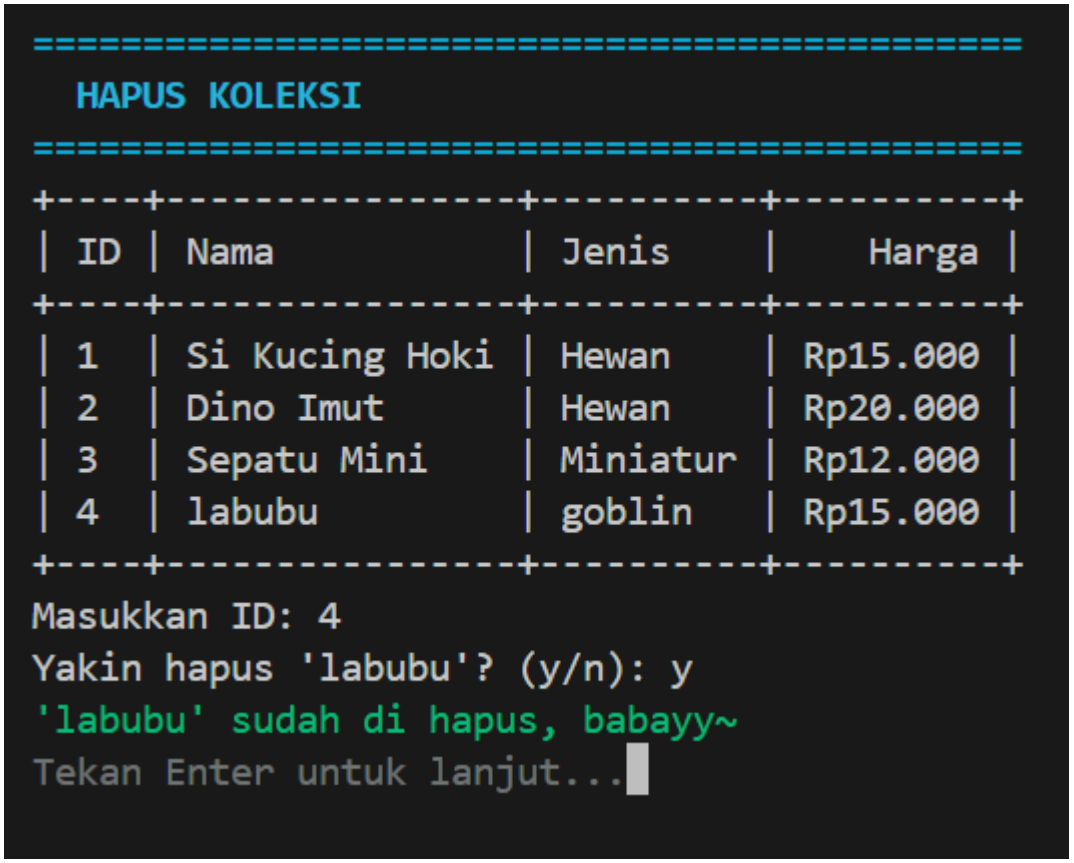
4.7 Output Menambahkan gantungan kunci



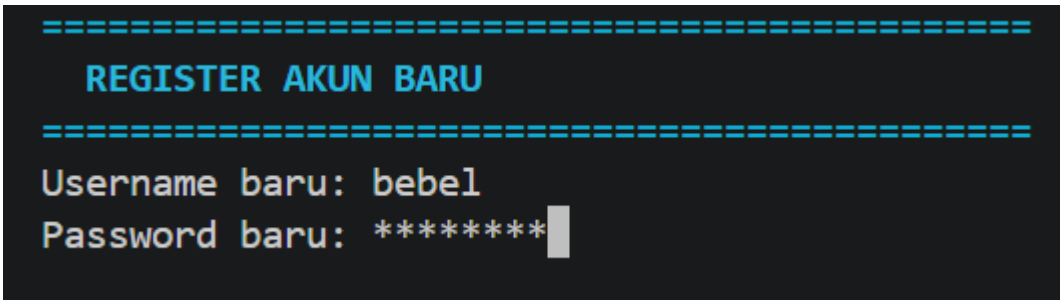
4.8 Output Menampilkan koleksi (admin)



4.9 Output Mengubah koleksi



4.10 Output Menghapus koleksi



4.10 Output Registrasi Akun baru

5. Langkah-langkah Git

5.1 Git Add

Git add digunakan untuk memasukkan file atau perubahan ke staging area sebelum disimpan ke dalam commit

```
PS C:\Users\amuha\OneDrive\Documents\PRAKTIKUM_APD_2026> git add .
PS C:\Users\amuha\OneDrive\Documents\PRAKTIKUM_APD_2026>
```

5.1 Git Add

5.2 Git Commit

Git commit digunakan untuk menyimpan perubahan yang sudah masuk staging area ke dalam repository lokal. Commit biasanya diberi pesan untuk menjelaskan perubahan yang dilakukan

```
PS C:\Users\amuha\OneDrive\Documents\PRAKTIKUM_APD_2026> git commit -m "POSTTEST 8"
[main 7a02e71] POSTTEST 8
 18 files changed, 381 insertions(+), 32 deletions(-)
 create mode 100644 posttest/posttest-apd-8/__pycache__/autentikasi.cpython-314.pyc
 create mode 100644 posttest/posttest-apd-8/__pycache__/data.cpython-314.pyc
 create mode 100644 posttest/posttest-apd-8/__pycache__/koleksi.cpython-314.pyc
 create mode 100644 posttest/posttest-apd-8/__pycache__/menu.cpython-314.pyc
 create mode 100644 posttest/posttest-apd-8/__pycache__/tampilan.cpython-314.pyc
 create mode 100644 posttest/posttest-apd-8/__pycache__/utilitas.cpython-314.pyc
 create mode 100644 posttest/posttest-apd-8/autentikasi.py
 create mode 100644 posttest/posttest-apd-8/data.py
 create mode 100644 posttest/posttest-apd-8/koleksi.py
 create mode 100644 posttest/posttest-apd-8/main.py
 create mode 100644 posttest/posttest-apd-8/menu.py
 create mode 100644 posttest/posttest-apd-8/tampilan.py
 create mode 100644 posttest/posttest-apd-8/utilitas.py
 delete mode 100644 proyek_akademik/__pycache__/autentikasi.cpython-314.pyc
 delete mode 100644 proyek_akademik/__pycache__/mahasiswa.cpython-314.pyc
 delete mode 100644 proyek_akademik/autentikasi.py
 delete mode 100644 proyek_akademik/mahasiswa.py
 delete mode 100644 proyek_akademik/main.py
PS C:\Users\amuha\OneDrive\Documents\PRAKTIKUM_APD_2026>
```

5.2 Git Commit

5.3 Git Push

Git push digunakan untuk mengirim commit dari repository lokal ke repository remote, misalnya Github.

```
PS C:\Users\amuha\OneDrive\Documents\PRAKTIKUM_APD_2026> git push -u
Enumerating objects: 25, done.
Counting objects: 100% (25/25), done.
Delta compression using up to 4 threads
Compressing objects: 100% (21/21), done.
Writing objects: 100% (21/21), 14.87 KiB | 211.00 KiB/s, done.
Total 21 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 1 local object.
To https://github.com/rulanam6/praktikum-apd-2026.git
   82f5967..886ebb8  main -> main
branch 'main' set up to track 'origin/main'.
PS C:\Users\amuha\OneDrive\Documents\PRAKTIKUM_APD_2026>
```

5.3 Git Push